11 통계적 머신러닝

강의 내용

Contents of Lecture

기간	내용	과제
01주차	오리엔테이션 Ipython - 파이썬에 날개를 달자	실습 과제 (채점 X)
02주차	Numpy 소개	실습 과제
03주차	Pandas로 데이터 가공하기	실습 과제
04주차	Matplotlib를 활용한 시각화	실습 과제
05주차	탐색적 데이터 분석	실습 과제
06주차	데이터의 표본분포	실습 과제
07주차	통계적 실험과 유의성 검정	실습 과제
08주차	중간고사 - 오프라인 오픈북으로 진행	기말 프로젝트 상세 공지

기간	내용	과제
09주차	인공지능 & 회귀/예측	실습 과제
10주차	회귀와 예측 / 분류	실습 과제
11주차	분류 및 머신러닝 기초	
12주차	**통계적 머신러닝**	**실습 과제**
13주차	비지도 학습	실습 과제
14주차	알고보면 쓸모있는 신기한 기계학습	프로젝트 레포트 제출
15주차	[기말고사] Team별 프로젝트 결과 발표	종강~!

1부: 통계적 머신러닝

1) k-최근접 이웃

2) 트리 모델

통계적 머신러닝

Statistical Machine Learning

- 최근 통계의 발전은 예측 모델링(회귀 및 분류 모두)을 위한 강력한 자동화 기술을 개발하는 데 전념
- 이는 통계적 기계 학습의 범주에 속하며 데이터 중심적이며 데이터에 선형 또는 기타 전체 구조를 부과하려고 하지 않는다는 점에서 고전적 통계 방법과 구별
- K-최근접 이웃 방법은 매우 간단, 유사한 레코드가 분류되는 방식에 따라 레코드를 분류
- 가장 성공적이고 널리 사용되는 기술은 의사 결정 트리에 적용된 앙상블 학습 기반
  - 앙상블 학습의 기본 아이디어는 단일 모델을 사용하는 것과는 대조적으로 많은 모델을 사용하여 예측을 형성하는 것
  - 의사 결정 트리는 예측 변수와 결과 변수 간의 관계에 대한 규칙을 학습하는 유연하고 자동적인 기술
  - 앙상블 학습과 의사 결정 트리의 조합이 최고의 성능을 발휘하는 기성 예측 모델링 기술로 이어짐
- 통계적 기계 학습에서 많은 기술의 발전은 캘리포니아 대학교 버클리의 통계학자 레오 브레이먼(Leo Breiman)과 스탠포드 대학교의 제리 프리드먼(Jerry Friedman)으로 거슬러 올라갈 수 있다
  - Berkeley 및 Stanford의 다른 연구원들의 작업과 함께 그들의 작업은 1984년 트리 모델의 개발로 시작
  - 1990년대에 배깅 및 부스팅의 앙상블 방법의 후속 개발은 통계적 기계 학습의 기초를 확립

통계적 머신러닝

Statistical Machine Learning

Figure 6-1. Leo Breiman, who was a professor of statistics at UC Berkeley, was at the forefront of the development of many techniques in a data scientist's toolkit today

Machine Learning Versus Statistics

In the context of predictive modeling, what is the difference between machine learning and statistics? There is not a bright line dividing the two disciplines. Machine learning tends to be focused more on developing efficient algorithms that scale to large data in order to optimize the predictive model. Statistics generally pays more attention to the probabilistic theory and underlying structure of the model. Bagging, and the random forest (see "Bagging and the Random Forest" on page 259), grew up firmly in the statistics camp. Boosting (see "Boosting" on page 270), on the other hand, has been developed in both disciplines but receives more attention on the machine learning side of the divide. Regardless of the history, the promise of boosting ensures that it will thrive as a technique in both statistics and machine learning.

K-최근접 이웃

K-Nearest Neighbors

K-Nearest Neighbors

The idea behind *K*-Nearest Neighbors (KNN) is very simple.[1] For each record to be classified or predicted:

1. Find *K* records that have similar features (i.e., similar predictor values).
2. For classification, find out what the majority class is among those similar records and assign that class to the new record.
3. For prediction (also called *KNN regression*), find the average among those similar records, and predict that average for the new record.

Key Terms for K-Nearest Neighbors

Neighbor
A record that has similar predictor values to another record.

Distance metrics
Measures that sum up in a single number how far one record is from another.

Standardization
Subtract the mean and divide by the standard deviation.

Synonym
Normalization

z-score
The value that results after standardization.

K
The number of neighbors considered in the nearest neighbor calculation.

KNN is one of the simpler prediction/classification techniques: there is no model to be fit (as in regression). This doesn't mean that using KNN is an automatic procedure. The prediction results depend on how the features are scaled, how similarity is measured, and how big *K* is set. Also, all predictors must be in numeric form. We will illustrate how to use the KNN method with a classification example.

K-최근접 이웃 > A Small Example: Predicting Loan Default

A Small Example: Predicting Loan Default

Table 6-1 shows a few records of personal loan data from LendingClub. LendingClub is a leader in peer-to-peer lending in which pools of investors make personal loans to individuals. The goal of an analysis would be to predict the outcome of a new potential loan: paid off versus default.

Table 6-1. A few records and columns for LendingClub loan data

Outcome	Loan amount	Income	Purpose	Years employed	Home ownership	State
Paid off	10000	79100	debt_consolidation	11	MORTGAGE	NV
Paid off	9600	48000	moving	5	MORTGAGE	TN
Paid off	18800	120036	debt_consolidation	11	MORTGAGE	MD
Default	15250	232000	small_business	9	MORTGAGE	CA
Paid off	17050	35000	debt_consolidation	4	RENT	MD
Paid off	5500	43000	debt_consolidation	4	RENT	KS

Consider a very simple model with just two predictor variables: `dti`, which is the ratio of debt payments (excluding mortgage) to income, and `payment_inc_ratio`, which is the ratio of the loan payment to income. Both ratios are multiplied by 100. Using a small set of 200 loans, `loan200`, with known binary outcomes (default or no-default, specified in the predictor `outcome200`), and with *K* set to 20, the KNN estimate for a new loan to be predicted, `newloan`, with `dti=22.5` and `payment_inc_ratio=9` can be calculated in *R* as follows:[2]

```
newloan <- loan200[1, 2:3, drop=FALSE]
knn_pred <- knn(train=loan200[-1, 2:3], test=newloan, cl=loan200[-1, 1], k=20)
knn_pred == 'paid off'
[1] TRUE
```

The KNN prediction is for the loan to default.

While *R* has a native `knn` function, the contributed *R* package `FNN`, for Fast Nearest Neighbor, scales more effectively to big data and provides more flexibility.

The `scikit-learn` package provides a fast and efficient implementation of KNN in *Python*:

```
predictors = ['payment_inc_ratio', 'dti']
outcome = 'outcome'

newloan = loan200.loc[0:0, predictors]
X = loan200.loc[1:, predictors]
y = loan200.loc[1:, outcome]

knn = KNeighborsClassifier(n_neighbors=20)
knn.fit(X, y)
knn.predict(newloan)
```

Figure 6-2 gives a visual display of this example. The new loan to be predicted is the cross in the middle. The squares (paid off) and circles (default) are the training data. The large black circle shows the boundary of the nearest 20 points. In this case, 9 defaulted loans lie within the circle, as compared with 11 paid-off loans. Hence the predicted outcome of the loan is paid off. Note that if we consider only three nearest neighbors, the prediction would be that the loan defaults.

K-최근접 이웃 > A Small Example: Predicting Loan Default

K-Nearest Neighbors

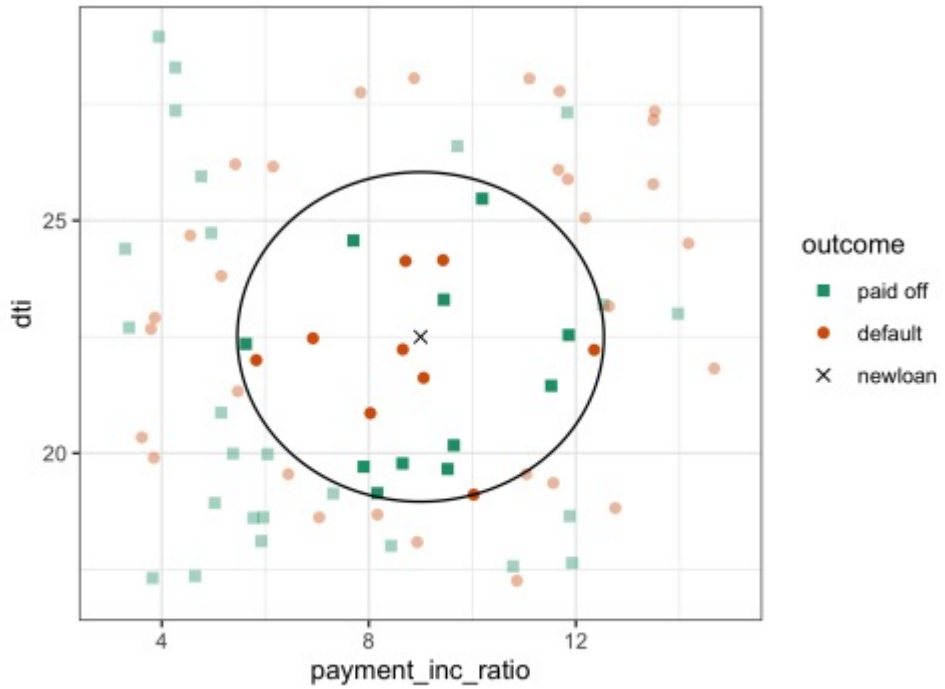


Figure 6-2. KNN prediction of loan default using two variables: debt-to-income ratio and loan-payment-to-income ratio

While the output of KNN for classification is typically a binary decision, such as default or paid off in the loan data, KNN routines usually offer the opportunity to output a probability (propensity) between 0 and 1. The probability is based on the fraction of one class in the *K* nearest neighbors. In the preceding example, this probability of default would have been estimated at $\frac{9}{20}$, or 0.45. Using a probability score lets you use classification rules other than simple majority votes (probability of 0.5). This is especially important in problems with imbalanced classes; see "Strategies for Imbalanced Data" on page 230. For example, if the goal is to identify members of a rare class, the cutoff would typically be set below 50%. One common approach is to set the cutoff at the probability of the rare event.

K-최근접 이웃 > Distance Metrics

Distance Metrics

Similarity (nearness) is determined using a *distance metric*, which is a function that measures how far two records $(x_1, x_2, \ldots, x_p)$ and $(u_1, u_2, \ldots, u_p)$ are from one another. The most popular distance metric between two vectors is *Euclidean distance*. To measure the Euclidean distance between two vectors, subtract one from the other, square the differences, sum them, and take the square root:

$$\sqrt{(x_1 - u_1)^2 + (x_2 - u_2)^2 + \cdots + (x_p - u_p)^2}.$$

Another common distance metric for numeric data is *Manhattan distance*:

$$|x_1 - u_1| + |x_2 - u_2| + \cdots + |x_p - u_p|$$

Euclidean distance corresponds to the straight-line distance between two points (e.g., as the crow flies). Manhattan distance is the distance between two points traversed in a single direction at a time (e.g., traveling along rectangular city blocks). For this reason, Manhattan distance is a useful approximation if similarity is defined as point-to-point travel time.

In measuring distance between two vectors, variables (features) that are measured with comparatively large scale will dominate the measure. For example, for the loan data, the distance would be almost solely a function of the income and loan amount variables, which are measured in tens or hundreds of thousands. Ratio variables would count for practically nothing in comparison. We address this problem by standardizing the data; see "Standardization (Normalization, z-Scores)" on page 243.

Other Distance Metrics

There are numerous other metrics for measuring distance between vectors. For numeric data, *Mahalanobis distance* is attractive since it accounts for the correlation between two variables. This is useful since if two variables are highly correlated, Mahalanobis will essentially treat these as a single variable in terms of distance. Euclidean and Manhattan distance do not account for the correlation, effectively placing greater weight on the attribute that underlies those features. Mahalanobis distance is the Euclidean distance between the principal components (see "Principal Components Analysis" on page 284). The downside of using Mahalanobis distance is increased computational effort and complexity; it is computed using the *covariance matrix* (see "Covariance Matrix" on page 202).

K-최근접 이웃 > One Hot Encoder

K-Nearest Neighbors

One Hot Encoder

The loan data in Table 6-1 includes several factor (string) variables. Most statistical and machine learning models require this type of variable to be converted to a series of binary dummy variables conveying the same information, as in Table 6-2. Instead of a single variable denoting the home occupant status as "owns with a mortgage," "owns with no mortgage," "rents," or "other," we end up with four binary variables. The first would be "owns with a mortgage—Y/N," the second would be "owns with no mortgage—Y/N," and so on. This one predictor, home occupant status, thus yields a vector with one 1 and three 0s that can be used in statistical and machine learning algorithms. The phrase *one hot encoding* comes from digital circuit terminology, where it describes circuit settings in which only one bit is allowed to be positive (hot).

Table 6-2. Representing home ownership factor data in Table 6-1 as a numeric dummy variable

OWNS_WITH_MORTGAGE	OWNS_WITHOUT_MORTGAGE	OTHER	RENT
1	0	0	0
1	0	0	0
1	0	0	0
1	0	0	0
0	0	0	1
0	0	0	1

In linear and logistic regression, one hot encoding causes problems with multicollinearity; see "Multicollinearity" on page 172. In such cases, one dummy is omitted (its value can be inferred from the other values). This is not an issue with KNN and other methods discussed in this book.

K-최근접 이웃 > Standardization (Normalization, z-Scores)

Standardization (Normalization, z-Scores)

In measurement, we are often not so much interested in "how much" but in "how different from the average." Standardization, also called *normalization*, puts all variables on similar scales by subtracting the mean and dividing by the standard deviation; in this way, we ensure that a variable does not overly influence a model simply due to the scale of its original measurement:

$$z = \frac{x - \bar{x}}{s}$$

The result of this transformation is commonly referred to as a *z-score*. Measurements are then stated in terms of "standard deviations away from the mean."

Normalization in this statistical context is not to be confused with *database normalization*, which is the removal of redundant data and the verification of data dependencies.

For KNN and a few other procedures (e.g., principal components analysis and clustering), it is essential to consider standardizing the data prior to applying the procedure. To illustrate this idea, KNN is applied to the loan data using `dti` and `payment_inc_ratio` (see "A Small Example: Predicting Loan Default" on page 239) plus two other variables: `revol_bal`, the total revolving credit available to the applicant in dollars, and `revol_util`, the percent of the credit being used. The new record to be predicted is shown here:

```
newloan
  payment_inc_ratio dti revol_bal revol_util
1            2.3932   1      1687        9.4
```

The magnitude of `revol_bal`, which is in dollars, is much bigger than that of the other variables. The `knn` function returns the index of the nearest neighbors as an attribute `nn.index`, and this can be used to show the top-five closest rows in `loan_df`:

```
loan_df <- model.matrix(~ -1 + payment_inc_ratio + dti + revol_bal +
                          revol_util, data=loan_data)
newloan <- loan_df[1, , drop=FALSE]
loan_df <- loan_df[-1,]
outcome <- loan_data[-1, 1]
knn_pred <- knn(train=loan_df, test=newloan, cl=outcome, k=5)
loan_df[attr(knn_pred, "nn.index"),]
```

```
       payment_inc_ratio  dti revol_bal revol_util
35537            1.47212 1.46      1686       10.0
33652            3.38178 6.37      1688        8.4
25864            2.36303 1.39      1691        3.5
42954            1.28160 7.14      1684        3.9
43600            4.12244 8.98      1684        7.2
```

K-최근접 이웃 > Standardization (Normalization, z-Scores)

Following the model fit, we can use the `kneighbors` method to identify the five closest rows in the training set with `scikit-learn`:

```
predictors = ['payment_inc_ratio', 'dti', 'revol_bal', 'revol_util']
outcome = 'outcome'

newloan = loan_data.loc[0:0, predictors]
X = loan_data.loc[1:, predictors]
y = loan_data.loc[1:, outcome]

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X, y)

nbrs = knn.kneighbors(newloan)
X.iloc[nbrs[1][0], :]
```

The value of `revol_bal` in these neighbors is very close to its value in the new record, but the other predictor variables are all over the map and essentially play no role in determining neighbors.

Compare this to KNN applied to the standardized data using the *R* function `scale`, which computes the *z*-score for each variable:

```
loan_df <- model.matrix(~ -1 + payment_inc_ratio + dti + revol_bal +
                          revol_util, data=loan_data)
loan_std <- scale(loan_df)
newloan_std <- loan_std[1, , drop=FALSE]
loan_std <- loan_std[-1,]
loan_df <- loan_df[-1,] ❶
outcome <- loan_data[-1, 1]
knn_pred <- knn(train=loan_std, test=newloan_std, cl=outcome, k=5)
loan_df[attr(knn_pred, "nn.index"),]
       payment_inc_ratio   dti  revol_bal  revol_util
2081            2.61091  1.03       1218         9.7
1439            2.34343  0.51        278         9.9
30216           2.71200  1.34       1075         8.5
28543           2.39760  0.74       2917         7.4
44738           2.34309  1.37        488         7.2
```

K-최근접 이웃 > Standardization (Normalization, z-Scores)

The `sklearn.preprocessing.StandardScaler` method is first trained with the predictors and is subsequently used to transform the data set prior to training the KNN model:

```
newloan = loan_data.loc[0:0, predictors]
X = loan_data.loc[1:, predictors]
y = loan_data.loc[1:, outcome]

scaler = preprocessing.StandardScaler()
scaler.fit(X * 1.0)

X_std = scaler.transform(X * 1.0)
newloan_std = scaler.transform(newloan * 1.0)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_std, y)

nbrs = knn.kneighbors(newloan_std)
X.iloc[nbrs[1][0], :]
```

The five nearest neighbors are much more alike in all the variables, providing a more sensible result. Note that the results are displayed on the original scale, but KNN was applied to the scaled data and the new loan to be predicted.

Using the *z*-score is just one way to rescale variables. Instead of the mean, a more robust estimate of location could be used, such as the median. Likewise, a different estimate of scale such as the interquartile range could be used instead of the standard deviation. Sometimes, variables are "squashed" into the 0–1 range. It's also important to realize that scaling each variable to have unit variance is somewhat arbitrary. This implies that each variable is thought to have the same importance in predictive power. If you have subjective knowledge that some variables are more important than others, then these could be scaled up. For example, with the loan data, it is reasonable to expect that the payment-to-income ratio is very important.

Normalization (standardization) does not change the distributional shape of the data; it does not make it normally shaped if it was not already normally shaped (see "Normal Distribution" on page 69).

K-최근접 이웃 > Choosing K

Choosing K

The choice of K is very important to the performance of KNN. The simplest choice is to set $K = 1$, known as the 1-nearest neighbor classifier. The prediction is intuitive: it is based on finding the data record in the training set most similar to the new record to be predicted. Setting $K = 1$ is rarely the best choice; you'll almost always obtain superior performance by using $K > 1$-nearest neighbors.

Generally speaking, if K is too low, we may be overfitting: including the noise in the data. Higher values of K provide smoothing that reduces the risk of overfitting in the training data. On the other hand, if K is too high, we may oversmooth the data and miss out on KNN's ability to capture the local structure in the data, one of its main advantages.

The K that best balances between overfitting and oversmoothing is typically determined by accuracy metrics and, in particular, accuracy with holdout or validation data. There is no general rule about the best K—it depends greatly on the nature of the data. For highly structured data with little noise, smaller values of K work best. Borrowing a term from the signal processing community, this type of data is sometimes referred to as having a high *signal-to-noise ratio* (*SNR*). Examples of data with a typically high SNR are data sets for handwriting and speech recognition. For noisy data with less structure (data with a low SNR), such as the loan data, larger values of K are appropriate. Typically, values of K fall in the range 1 to 20. Often, an odd number is chosen to avoid ties.

Bias-Variance Trade-off

The tension between oversmoothing and overfitting is an instance of the *bias-variance trade-off*, a ubiquitous problem in statistical model fitting. Variance refers to the modeling error that occurs because of the choice of training data; that is, if you were to choose a different set of training data, the resulting model would be different. Bias refers to the modeling error that occurs because you have not properly identified the underlying real-world scenario; this error would not disappear if you simply added more training data. When a flexible model is overfit, the variance increases. You can reduce this by using a simpler model, but the bias may increase due to the loss of flexibility in modeling the real underlying situation. A general approach to handling this trade-off is through *cross-validation*. See "Cross-Validation" on page 155 for more details.

K-최근접 이웃 > KNN as a Feature Engine

KNN as a Feature Engine

KNN gained its popularity due to its simplicity and intuitive nature. In terms of performance, KNN by itself is usually not competitive with more sophisticated classification techniques. In practical model fitting, however, KNN can be used to add "local knowledge" in a staged process with other classification techniques:

1. KNN is run on the data, and for each record, a classification (or quasi-probability of a class) is derived.
2. That result is added as a new feature to the record, and another classification method is then run on the data. The original predictor variables are thus used twice.

At first you might wonder whether this process, since it uses some predictors twice, causes a problem with multicollinearity (see "Multicollinearity" on page 172). This is not an issue, since the information being incorporated into the second-stage model is highly local, derived only from a few nearby records, and is therefore additional information and not redundant.

You can think of this staged use of KNN as a form of ensemble learning, in which multiple predictive modeling methods are used in conjunction with one another. It can also be considered as a form of feature engineering in which the aim is to derive features (predictor variables) that have predictive power. Often this involves some manual review of the data; KNN gives a fairly automatic way to do this.

For example, consider the King County housing data. In pricing a home for sale, a realtor will base the price on similar homes recently sold, known as "comps." In essence, realtors are doing a manual version of KNN: by looking at the sale prices of similar homes, they can estimate what a home will sell for. We can create a new feature for a statistical model to mimic the real estate professional by applying KNN to recent sales. The predicted value is the sales price, and the existing predictor variables could include location, total square feet, type of structure, lot size, and number of bedrooms and bathrooms. The new predictor variable (feature) that we add via KNN is the KNN predictor for each record (analogous to the realtors' comps). Since we are predicting a numerical value, the average of the *K*-Nearest Neighbors is used instead of a majority vote (known as *KNN regression*).

K-최근접 이웃 > KNN as a Feature Engine

Similarly, for the loan data, we can create features that represent different aspects of the loan process. For example, the following *R* code would build a feature that represents a borrower's creditworthiness:

```
borrow_df <- model.matrix(~ -1 + dti + revol_bal + revol_util + open_acc +
                          delinq_2yrs_zero + pub_rec_zero, data=loan_data)
borrow_knn <- knn(borrow_df, test=borrow_df, cl=loan_data[, 'outcome'],
                  prob=TRUE, k=20)
prob <- attr(borrow_knn, "prob")
borrow_feature <- ifelse(borrow_knn == 'default', prob, 1 - prob)
summary(borrow_feature)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  0.000   0.400   0.500   0.501   0.600   0.950
```

With `scikit-learn`, we use the `predict_proba` method of the trained model to get the probabilities:

```
predictors = ['dti', 'revol_bal', 'revol_util', 'open_acc',
              'delinq_2yrs_zero', 'pub_rec_zero']
outcome = 'outcome'

X = loan_data[predictors]
y = loan_data[outcome]

knn = KNeighborsClassifier(n_neighbors=20)
knn.fit(X, y)

loan_data['borrower_score'] = knn.predict_proba(X)[:, 1]
loan_data['borrower_score'].describe()
```

The result is a feature that predicts the likelihood a borrower will default based on his credit history.

Key Ideas

- *K*-Nearest Neighbors (KNN) classifies a record by assigning it to the class that similar records belong to.
- Similarity (distance) is determined by Euclidian distance or other related metrics.
- The number of nearest neighbors to compare a record to, *K*, is determined by how well the algorithm performs on training data, using different values for *K*.
- Typically, the predictor variables are standardized so that variables of large scale do not dominate the distance metric.
- KNN is often used as a first stage in predictive modeling, and the predicted value is added back into the data as a *predictor* for second-stage (non-KNN) modeling.

Tree Models

Tree models, also called *Classification and Regression Trees* (*CART*),[3] *decision trees*, or just *trees*, are an effective and popular classification (and regression) method initially developed by Leo Breiman and others in 1984. Tree models, and their more powerful descendants *random forests* and *boosted trees* (see "Bagging and the Random Forest" on page 259 and "Boosting" on page 270), form the basis for the most widely used and powerful predictive modeling tools in data science for regression and classification.

Key Terms for Trees

Recursive partitioning
Repeatedly dividing and subdividing the data with the goal of making the outcomes in each final subdivision as homogeneous as possible.

Split value
A predictor value that divides the records into those where that predictor is less than the split value, and those where it is more.

Node
In the decision tree, or in the set of corresponding branching rules, a node is the graphical or rule representation of a split value.

Leaf
The end of a set of if-then rules, or branches of a tree—the rules that bring you to that leaf provide one of the classification rules for any record in a tree.

Loss
The number of misclassifications at a stage in the splitting process; the more losses, the more impurity.

Impurity
The extent to which a mix of classes is found in a subpartition of the data (the more mixed, the more impure).

Synonym
Heterogeneity

Antonyms
Homogeneity, purity

Pruning
The process of taking a fully grown tree and progressively cutting its branches back to reduce overfitting.

A tree model is a set of "if-then-else" rules that are easy to understand and to implement. In contrast to linear and logistic regression, trees have the ability to discover hidden patterns corresponding to complex interactions in the data. However, unlike KNN or naive Bayes, simple tree models can be expressed in terms of predictor relationships that are easily interpretable.

Decision Trees in Operations Research

The term *decision trees* has a different (and older) meaning in decision science and operations research, where it refers to a human decision analysis process. In this meaning, decision points, possible outcomes, and their estimated probabilities are laid out in a branching diagram, and the decision path with the maximum expected value is chosen.

트리 모델 > A Simple Example

A Simple Example

The two main packages to fit tree models in *R* are `rpart` and `tree`. Using the `rpart` package, a model is fit to a sample of 3,000 records of the loan data using the variables `payment_inc_ratio` and `borrower_score` (see "K-Nearest Neighbors" on page 238 for a description of the data):

```
library(rpart)
loan_tree <- rpart(outcome ~ borrower_score + payment_inc_ratio,
                   data=loan3000, control=rpart.control(cp=0.005))
plot(loan_tree, uniform=TRUE, margin=0.05)
text(loan_tree)
```

The `sklearn.tree.DecisionTreeClassifier` provides an implementation of a decision tree. The `dmba` package provides a convenience function to create a visualization inside a Jupyter notebook:

```
predictors = ['borrower_score', 'payment_inc_ratio']
outcome = 'outcome'

X = loan3000[predictors]
y = loan3000[outcome]

loan_tree = DecisionTreeClassifier(random_state=1, criterion='entropy',
                                   min_impurity_decrease=0.003)
loan_tree.fit(X, y)
plotDecisionTree(loan_tree, feature_names=predictors,
                 class_names=loan_tree.classes_)
```

The resulting tree is shown in Figure 6-3. Due to the different implementations, you will find that the results from *R* and *Python* are not identical; this is expected. These classification rules are determined by traversing through a hierarchical tree, starting at the root and moving left if the node is true and right if not, until a leaf is reached.

Typically, the tree is plotted upside-down, so the root is at the top and the leaves are at the bottom. For example, if we get a loan with `borrower_score` of 0.6 and a `payment_inc_ratio` of 8.0, we end up at the leftmost leaf and predict the loan will be paid off.

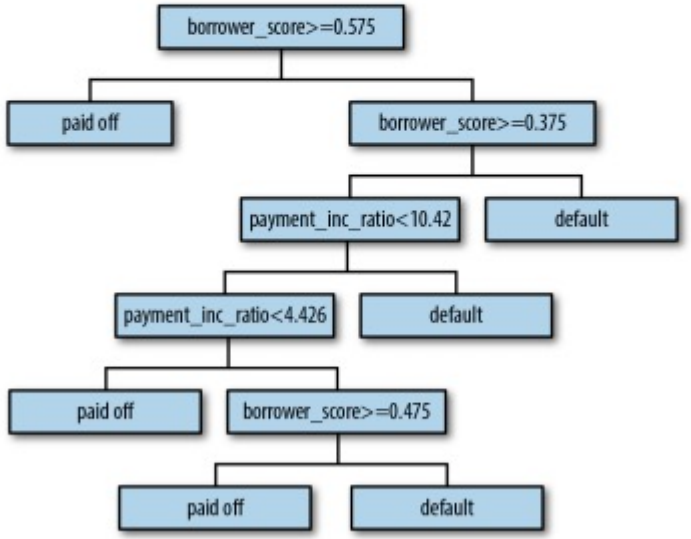


Figure 6-3. The rules for a simple tree model fit to the loan data

트리 모델 > A Simple Example

A nicely printed version of the tree is also easily produced in *R*:

```
loan_tree
n= 3000

node), split, n, loss, yval, (yprob)
      * denotes terminal node

1) root 3000 1445 paid off (0.5183333 0.4816667)
  2) borrower_score>=0.575 878  261 paid off (0.7027335 0.2972665) *
  3) borrower_score< 0.575 2122  938 default (0.4420358 0.5579642)
    6) borrower_score>=0.375 1639  802 default (0.4893228 0.5106772)
      12) payment_inc_ratio< 10.42265 1157  547 paid off (0.5272256 0.4727744)
        24) payment_inc_ratio< 4.42601 334  139 paid off (0.5838323 0.4161677) *
        25) payment_inc_ratio>=4.42601 823  408 paid off (0.5042527 0.4957473)
          50) borrower_score>=0.475 418  190 paid off (0.5454545 0.4545455) *
          51) borrower_score< 0.475 405  187 default (0.4617284 0.5382716) *
      13) payment_inc_ratio>=10.42265 482  192 default (0.3983402 0.6016598) *
    7) borrower_score< 0.375 483  136 default (0.2815735 0.7184265) *
```

The depth of the tree is shown by the indent. Each node corresponds to a provisional classification determined by the prevalent outcome in that partition. The "loss" is the number of misclassifications yielded by the provisional classification in a partition. For example, in node 2, there were 261 misclassifications out of a total of 878 total records. The values in the parentheses correspond to the proportion of records that are paid off or in default, respectively. For example, in node 13, which predicts default, over 60 percent of the records are loans that are in default.

The `scikit-learn` documentation describes how to create a text representation of a decision tree model. We included a convenience function in our `dmba` package:

```
print(textDecisionTree(loan_tree))
--
node=0 test node: go to node 1 if 0 <= 0.5750000178813934 else to node 6
  node=1 test node: go to node 2 if 0 <= 0.32500000298023224 else to node 3
    node=2 leaf node: [[0.785, 0.215]]
    node=3 test node: go to node 4 if 1 <= 10.42264986038208 else to node 5
      node=4 leaf node: [[0.488, 0.512]]
      node=5 leaf node: [[0.613, 0.387]]
  node=6 test node: go to node 7 if 1 <= 9.19082498550415 else to node 10
    node=7 test node: go to node 8 if 0 <= 0.7249999940395355 else to node 9
      node=8 leaf node: [[0.247, 0.753]]
      node=9 leaf node: [[0.073, 0.927]]
    node=10 leaf node: [[0.457, 0.543]]
```

The Recursive Partitioning Algorithm

The algorithm to construct a decision tree, called *recursive partitioning*, is straightforward and intuitive. The data is repeatedly partitioned using predictor values that do the best job of separating the data into relatively homogeneous partitions. Figure 6-4 shows the partitions created for the tree in Figure 6-3. The first rule, depicted by rule 1, is `borrower_score >= 0.575` and segments the right portion of the plot. The second rule is `borrower_score < 0.375` and segments the left portion.

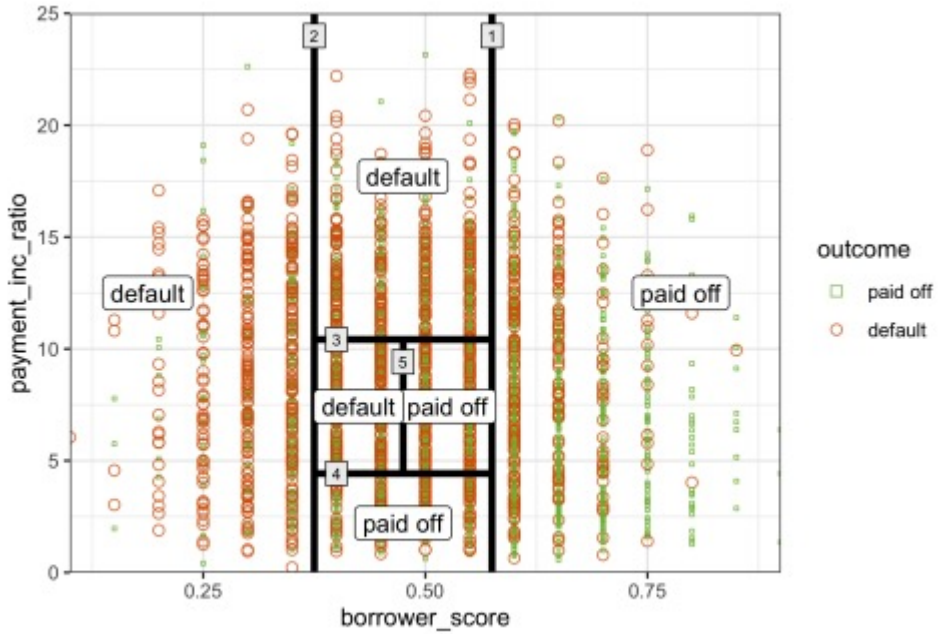


Figure 6-4. The first three rules for a simple tree model fit to the loan data

Suppose we have a response variable Y and a set of P predictor variables X_j for $j = 1, \cdots, P$. For a partition A of records, recursive partitioning will find the best way to partition A into two subpartitions:

1. For each predictor variable X_j:
   a. For each value s_j of X_j:
      i. Split the records in A with X_j values $< s_j$ as one partition, and the remaining records where $X_j \geq s_j$ as another partition.
      ii. Measure the homogeneity of classes within each subpartition of A.
   b. Select the value of s_j that produces maximum within-partition homogeneity of class.
2. Select the variable X_j and the split value s_j that produces maximum within-partition homogeneity of class.

Now comes the recursive part:

1. Initialize A with the entire data set.
2. Apply the partitioning algorithm to split A into two subpartitions, A_1 and A_2.
3. Repeat step 2 on subpartitions A_1 and A_2.
4. The algorithm terminates when no further partition can be made that sufficiently improves the homogeneity of the partitions.

The end result is a partitioning of the data, as in Figure 6-4, except in P-dimensions, with each partition predicting an outcome of 0 or 1 depending on the majority vote of the response in that partition.

In addition to a binary 0/1 prediction, tree models can produce a probability estimate based on the number of 0s and 1s in the partition. The estimate is simply the sum of 0s or 1s in the partition divided by the number of observations in the partition:

$$\text{Prob}(Y = 1) = \frac{\text{Number of 1s in the partition}}{\text{Size of the partition}}$$

The estimated Prob($Y = 1$) can then be converted to a binary decision; for example, set the estimate to 1 if Prob($Y = 1$) > 0.5.

트리 모델 > Measuring Homogeneity or Impurity

Measuring Homogeneity or Impurity

Tree models recursively create partitions (sets of records), A, that predict an outcome of $Y = 0$ or $Y = 1$. You can see from the preceding algorithm that we need a way to measure homogeneity, also called *class purity*, within a partition. Or equivalently, we need to measure the impurity of a partition. The accuracy of the predictions is the proportion p of misclassified records within that partition, which ranges from 0 (perfect) to 0.5 (purely random guessing).

It turns out that accuracy is not a good measure for impurity. Instead, two common measures for impurity are the *Gini impurity* and *entropy* of *information*. While these (and other) impurity measures apply to classification problems with more than two classes, we focus on the binary case. The Gini impurity for a set of records A is:

$$I(A) = p(1 - p)$$

The entropy measure is given by:

$$I(A) = -p \log_2 (p) - (1 - p) \log_2 (1 - p)$$

Figure 6-5 shows that Gini impurity (rescaled) and entropy measures are similar, with entropy giving higher impurity scores for moderate and high accuracy rates.

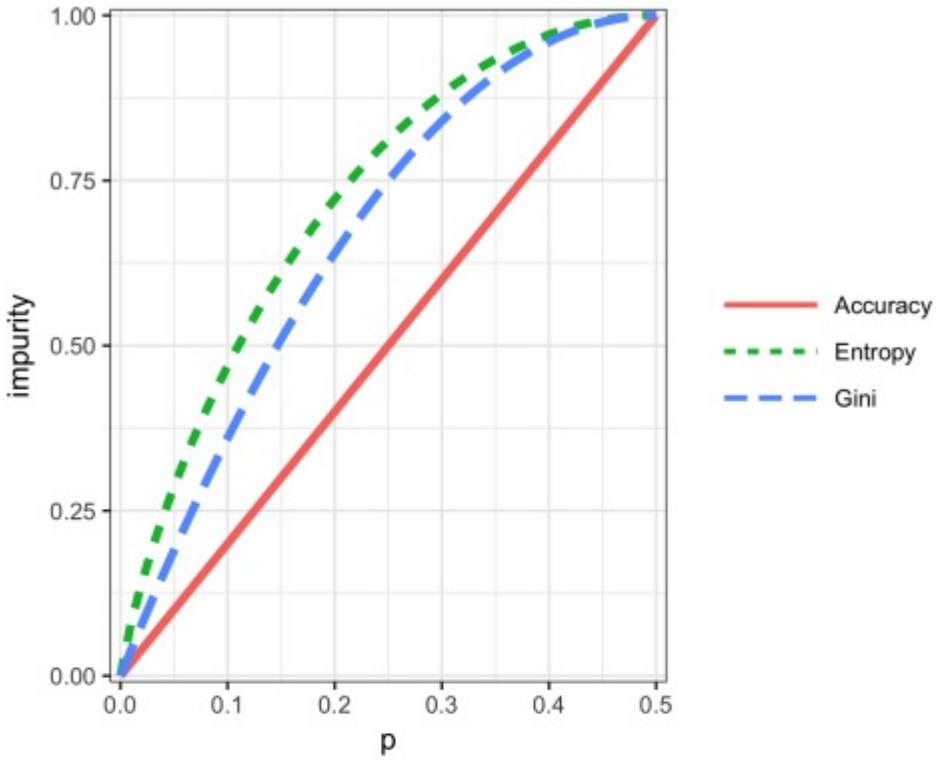


Figure 6-5. Gini impurity and entropy measures

Gini Coefficient

Gini impurity is not to be confused with the *Gini coefficient*. They represent similar concepts, but the Gini coefficient is limited to the binary classification problem and is related to the AUC metric (see "AUC" on page 226).

The impurity metric is used in the splitting algorithm described earlier. For each proposed partition of the data, impurity is measured for each of the partitions that result from the split. A weighted average is then calculated, and whichever partition (at each stage) yields the lowest weighted average is selected.

트리 모델 > Stopping the Tree from Growing

Stopping the Tree from Growing

As the tree grows bigger, the splitting rules become more detailed, and the tree gradually shifts from identifying "big" rules that identify real and reliable relationships in the data to "tiny" rules that reflect only noise. A fully grown tree results in completely pure leaves and, hence, 100% accuracy in classifying the data that it is trained on. This accuracy is, of course, illusory—we have overfit (see "Bias-Variance Trade-off" on page 247) the data, fitting the noise in the training data, not the signal that we want to identify in new data.

We need some way to determine when to stop growing a tree at a stage that will generalize to new data. There are various ways to stop splitting in *R* and *Python*:

- Avoid splitting a partition if a resulting subpartition is too small, or if a terminal leaf is too small. In `rpart` (*R*), these constraints are controlled separately by the parameters `minsplit` and `minbucket`, respectively, with defaults of `20` and `7`. In *Python*'s `DecisionTreeClassifier`, we can control this using the parameters `min_samples_split` (default `2`) and `min_samples_leaf` (default `1`).
- Don't split a partition if the new partition does not "significantly" reduce the impurity. In `rpart`, this is controlled by the *complexity parameter* `cp`, which is a measure of how complex a tree is—the more complex, the greater the value of `cp`. In practice, `cp` is used to limit tree growth by attaching a penalty to additional complexity (splits) in a tree. `DecisionTreeClassifier` (*Python*) has the parameter `min_impurity_decrease`, which limits splitting based on a weighted impurity decrease value. Here, smaller values will lead to more complex trees.

These methods involve arbitrary rules and can be useful for exploratory work, but we can't easily determine optimum values (i.e., values that maximize predictive accuracy with new data). We need to combine cross-validation with either systematically changing the model parameters or modifying the tree through pruning.

Controlling tree complexity in *R*

With the complexity parameter, `cp`, we can estimate what size tree will perform best with new data. If `cp` is too small, then the tree will overfit the data, fitting noise and not signal. On the other hand, if `cp` is too large, then the tree will be too small and have little predictive power. The default in `rpart` is 0.01, although for larger data sets, you are likely to find this is too large. In the previous example, `cp` was set to `0.005` since the default led to a tree with a single split. In exploratory analysis, it is sufficient to simply try a few values.

Determining the optimum `cp` is an instance of the bias-variance trade-off. The most common way to estimate a good value of `cp` is via cross-validation (see "Cross-Validation" on page 155):

1. Partition the data into training and validation (holdout) sets.
2. Grow the tree with the training data.
3. Prune it successively, step by step, recording `cp` (using the *training* data) at each step.
4. Note the `cp` that corresponds to the minimum error (loss) on the *validation* data.
5. Repartition the data into training and validation, and repeat the growing, pruning, and `cp` recording process.
6. Do this again and again, and average the `cp`s that reflect minimum error for each tree.
7. Go back to the original data, or future data, and grow a tree, stopping at this optimum `cp` value.

In `rpart`, you can use the argument `cptable` to produce a table of the `cp` values and their associated cross-validation error (`xerror` in *R*), from which you can determine the `cp` value that has the lowest cross-validation error.

Controlling tree complexity in *Python*

Neither the complexity parameter nor pruning is available in `scikit-learn`'s decision tree implementation. The solution is to use grid search over combinations of different parameter values. For example, we can vary `max_depth` in the range 5 to 30 and `min_samples_split` between 20 and 100. The `GridSearchCV` method in `scikit-learn` is a convenient way to combine the exhaustive search through all combinations with cross-validation. An optimal parameter set is then selected using the cross-validated model performance.

Predicting a Continuous Value

Predicting a continuous value (also termed *regression*) with a tree follows the same logic and procedure, except that impurity is measured by squared deviations from the mean (squared errors) in each subpartition, and predictive performance is judged by the square root of the mean squared error (RMSE) (see "Assessing the Model" on page 153) in each partition.

`scikit-learn` has the `sklearn.tree.DecisionTreeRegressor` method to train a decision tree regression model.

트리 모델 > How Trees Are Used

How Trees Are Used

One of the big obstacles faced by predictive modelers in organizations is the perceived "black box" nature of the methods they use, which gives rise to opposition from other elements of the organization. In this regard, the tree model has two appealing aspects:

- Tree models provide a visual tool for exploring the data, to gain an idea of what variables are important and how they relate to one another. Trees can capture nonlinear relationships among predictor variables.
- Tree models provide a set of rules that can be effectively communicated to non-specialists, either for implementation or to "sell" a data mining project.

When it comes to prediction, however, harnessing the results from multiple trees is typically more powerful than using just a single tree. In particular, the random forest and boosted tree algorithms almost always provide superior predictive accuracy and performance (see "Bagging and the Random Forest" on page 259 and "Boosting" on page 270), but the aforementioned advantages of a single tree are lost.

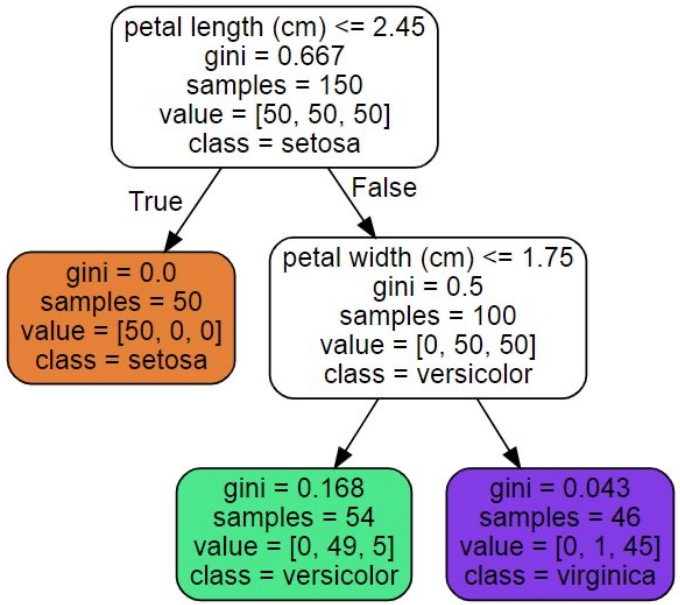
Key Ideas

- Decision trees produce a set of rules to classify or predict an outcome.
- The rules correspond to successive partitioning of the data into subpartitions.
- Each partition, or split, references a specific value of a predictor variable and divides the data into records where that predictor value is above or below that split value.
- At each stage, the tree algorithm chooses the split that minimizes the outcome impurity within each subpartition.
- When no further splits can be made, the tree is fully grown and each terminal node, or leaf, has records of a single class; new cases following that rule (split) path would be assigned that class.
- A fully grown tree overfits the data and must be pruned back so that it captures signal and not noise.
- Multiple-tree algorithms like random forests and boosted trees yield better predictive performance, but they lose the rule-based communicative power of single trees.

결정트리

Decision Tree

결정 트리의 예시



결정트리 세부 개념

- 루트 노드(Root Node): 깊이가 0인 맨 꼭대기의 노드
- 리프 노드(Leaf Node): 자식 노드를 가지지 않는 노드
- samples 속성: 얼마나 많은 훈련 샘플이 적용되었는지
- value 속성: 노드에서 각 클래스에 얼마나 많은 훈련 샘플이 있는지
- Gini 속성: **불순도**를 측정
- 즉, 한 노드의 모든 샘플이 같은 클래스에 속해 있다면, 이 노드를 순수(gini = 0)하다고 한다.
- ex) 위 그림에서, 깊이 2의 왼쪽 노드의 gini 점수는

$$1 - \frac{0}{54}^2 - \frac{49}{54}^2 - \frac{5}{54}^2 = 0.168$$

식 6-1 지니 불순도

$$G_i = 1 - \sum_{k=1}^{n} p_{i,k}^2$$

• 이 식에서 $p_{i,k}$는 i번째 노드에 있는 훈련 샘플 중 클래스 k에 속한 샘플의 비율입니다.

결정트리

Decision Tree

결정 트리의 장점

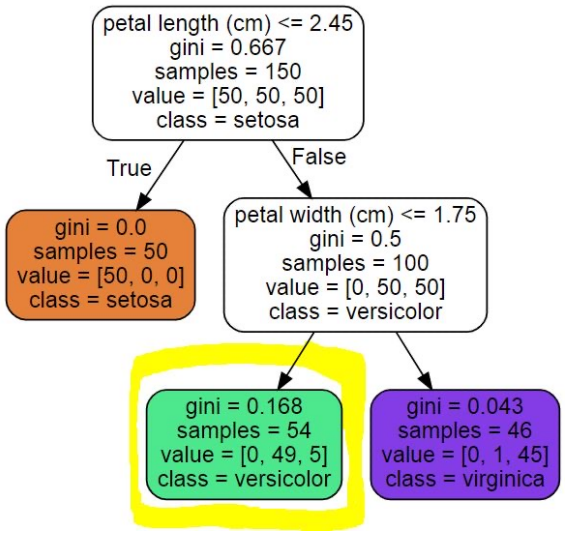
- 결정 트리의 장점은 데이터 전처리가 거의 필요하지 않다는 점이다.
- 특성의 스케일을 맞추거나 평균을 원점에 맞추는 작업이 필요하지 않다. (즉, feature scaling 불필요)
- 모델 해석능력이 높다 (**화이트 박스** VS 블랙 박스)
  - 화이트 박스 모델: 직관적이고 결정 방식을 이해하기 쉽다.
    - ex) **결정 트리**
  - 블랙 박스 모델: 성능이 뛰어나고 예측을 만드는 연산 과정을 쉽게 확인할 수 있으나, 왜 그러한 예측을 만드는지를 설명하기 어렵다.
    - ex) 랜덤 포레스트, 신경망

결정트리 – 클래스 확률 추정

Decision Tree

결정 트리의 클래스 확률 추정

- 결정 트리는 한 샘플이 특정 클래스 k에 속할 확률을 추정할 수도 있다.
- 예를 들어, 길이가 5cm이고 너비가 1.5cm인 꽃잎을 발견했다고 가정해보자.
  - 이에 해당하는 리프 노드(Leaf Node)는 깊이 2에서 왼쪽 노드이다.
    (즉, 노란색 네모 박스 부분)
  - 이 때, 각 클래스에 해당하는 확률을 계산해보면 다음과 같다.
    - Iris-Setosa: 0% (= 0 / 54)
    - Iris-Versicolor: 90.7% (= 49 / 54)
    - Iris-Virginica: 9.3% (= 5 / 54)




```
In [5]: tree_clf.predict_proba([[5, 1.5]])
Out[5]: array([[0.        , 0.90740741, 0.09259259]])
```

결정트리 – CART 훈련 알고리즘

Decision Tree

CART (Classification and Regression Tree)

- 먼저 훈련 세트를 하나의 특성 k의 임곗값 t_k를 사용해 두 개의 subset으로 나눈다.
- 그 다음, (크기에 따른 가중치가 적용된) 가장 순수한(gini = 0에 가까운) subset으로 나눌 수 있는 (k, t_k) 짝을 찾는다.
- 따라서 CART 알고리즘이 최소화해야 하는 비용 함수는 다음과 같다.
- CART 알고리즘이 subset을 나누는 과정은 (max_depth 매개변수로 정의된) 최대 깊이가 되면 중지하거나, 불순도를 줄이는 분할을 찾을 수 없을 때 멈추게 된다.
  - max_depth 외에도 min_samples_split, min_samples_leaf, min_weight_fraction_leaf, max_leaf_nodes와 같은 매개변수도 중지 조건에 관여한다.
- CART 알고리즘은 탐욕적 알고리즘(greedy algorithm)이다.
  - 탐욕적 알고리즘은 **항상 최적의 솔루션을 보장하지 않는다.**
  - 또한 **최적의 트리를 찾는 것은 NP-완전(NP-Complete) 문제로도 알려져 있다.**
    - 그러므로 "**납득할 만한 좋은 솔루션**"으로만 만족을 해야 한다.

식 6-2 분류에 대한 CART 비용 함수

$$J(k, t_k) = \frac{m_{\text{left}}}{m} G_{\text{left}} + \frac{m_{\text{right}}}{m} G_{\text{right}}$$

여기서 $G_{\text{left/right}}$는 왼쪽/오른쪽 서브셋의 불순도

$m_{\text{left/right}}$는 왼쪽/오른쪽 서브셋의 샘플 수

결정트리 – 계산 복잡도 X 규제 매개변수

Decision Tree

계산 복잡도

- 결정 트리를 탐색하기 위해서는 약 O(log_2(m))개의 노드를 거쳐야 한다.
  - 각 노드는 하나의 특성 값만 확인하기 때문에, 예측에 필요한 전체 복잡도는 특성 수와 무관하게 O(log_2(m))이다.
  - 따라서 큰 훈련 세트를 다룰 때도 예측 속도가 매우 빠르다.

규제 매개변수

- 결정 트리는 훈련 데이터에 대한 제약 사항이 거의 없다. (즉, 과대적합되기 쉽다)
- 결정 트리는 훈련되기 전에 파라미터 수가 결정되지 않기 때문에 **Nonparametric Model**이라고 한다.
- 훈련 데이터에 대한 과대적합을 피하기 위해, 모델 학습 시에 결정 트리의 자유도를 제한할 필요가 있다. (규제)

결정트리 – 규제 매개변수

Decision Tree

규제 조절 매개변수

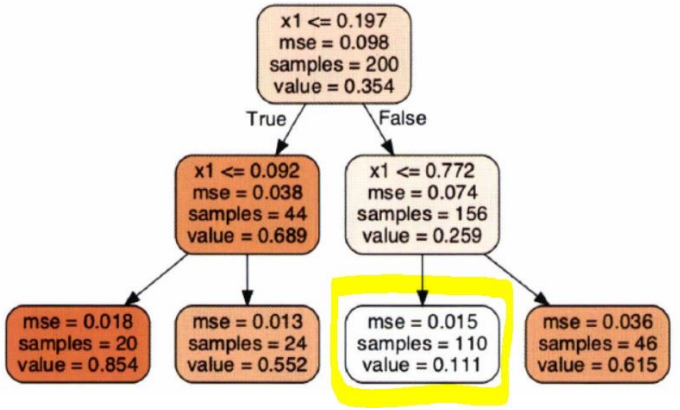
- min_으로 시작하는 매개변수를 증가시키거나, max_로 시작하는 매개변수를 감소시키면 모델에 대한 규제가 커진다.
- max_depth: 결정 트리의 최대 깊이
  - max_depth를 감소시키면 모델에 대한 규제가 커진다. (즉, **과대적합** 위험 감소)
- min_samples_split: 분할되기 위해 노드가 가져야 하는 최소 샘플 개수
  - min_samples_split을 증가시키면 모델에 대한 규제가 커진다. (즉, **과대적합** 위험 감소)
- min_samples_leaf: 리프 노드가 가지고 있어야 할 최소 샘플 개수
  - min_samples_leaf를 증가시키면 모델에 대한 규제가 커진다. (즉, **과대적합** 위험 감소)
- min_weight_fraction_leaf: min_samples_leaf와 같지만, 가중치가 부여된 전체 샘플 수에서의 비율
  - min_weight_fraction_leaf를 증가시키면 모델에 대한 규제가 커진다. (즉, **과대적합** 위험 감소)
- max_leaf_nodes: 리프 노드의 최대 개수
  - max_leaf_nodes를 감소시키면 모델에 대한 규제가 커진다. (즉, **과대적합** 위험 감소)
- max_features: 각 노드에서 분할에 사용할 특성의 최대 개수
  - max_features를 감소시키면 모델에 대한 규제가 커진다. (즉, **과대적합** 위험 감소)

결정트리 – 회귀분석

Decision Tree

결정트리 기반 회귀분석

- 사이킷런의 DecisionTreeRegressor를 사용한다.
- 분류 트리와의 주요한 차이점은 각 노드에서 클래스를 예측하는 대신 어떤 값을 예측한다는 점이다.
- 가령, 아래의 회귀 결정 트리에서 x1 = 0.6인 샘플의 클래스를 예측한다고 가정해보자.
  - 루트 노드부터 시작해서 트리를 순회하면, 결국 value = 0.111인 리프 노드에 도달할 것이다.
  - 이 리프 노드에 있는 110개의 훈련 샘플의 평균 target 값이 예측값이 된다.
  - 그리고 이 예측값을 사용하여 110개 샘플에 대한 평균제곱오차(MSE)를 계산하면 0.015가 된다.
- 정리하면, 각 영역의 예측값은 항상 그 영역에 있는 target 값의 평균이 된다.
- 알고리즘은 예측값과 가능한 한 많은 샘플이 가까이 있도록 영역을 분할한다.
  - 즉, 훈련 세트를 불순도를 최소화하는 방향으로 분할하는 대신, 평균제곱오차(MSE)를 최소화하도록 분할한다.
  - CART 알고리즘이 최소화하기 위한 비용 함수는 다음과 같다.



식 6-4 회귀를 위한 CART 비용 함수

$$J(k, t_k) = \frac{m_{\text{left}}}{m}\text{MSE}_{\text{left}} + \frac{m_{\text{right}}}{m}\text{MSE}_{\text{right}}$$

여기서

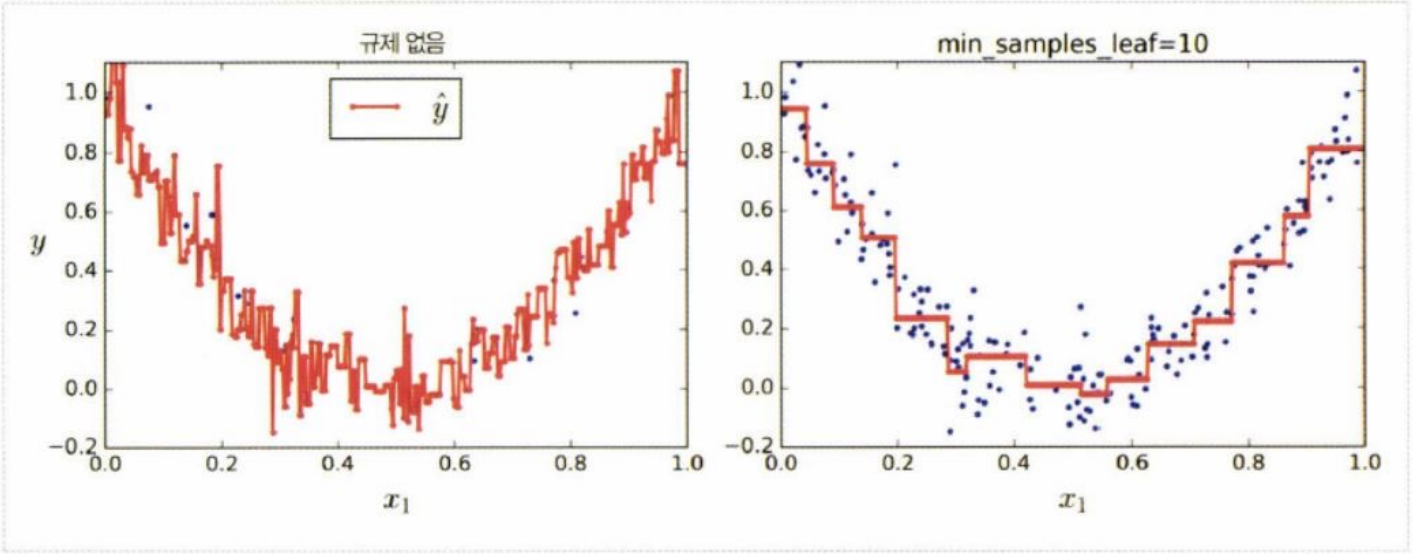
$$\begin{cases} \text{MSE}_{\text{node}} = \sum_{i \in \text{node}} (\hat{y}_{\text{node}} - y^{(i)})^2 \\ \hat{y}_{\text{node}} = \frac{1}{m_{\text{node}}} \sum_{i \in \text{node}} y^{(i)} \end{cases}$$

결정트리 – 회귀분석

Decision Tree

회귀분석 규제 적용시

- 시각화로부터 확인되는 것처럼 과대적합이 개선된다.



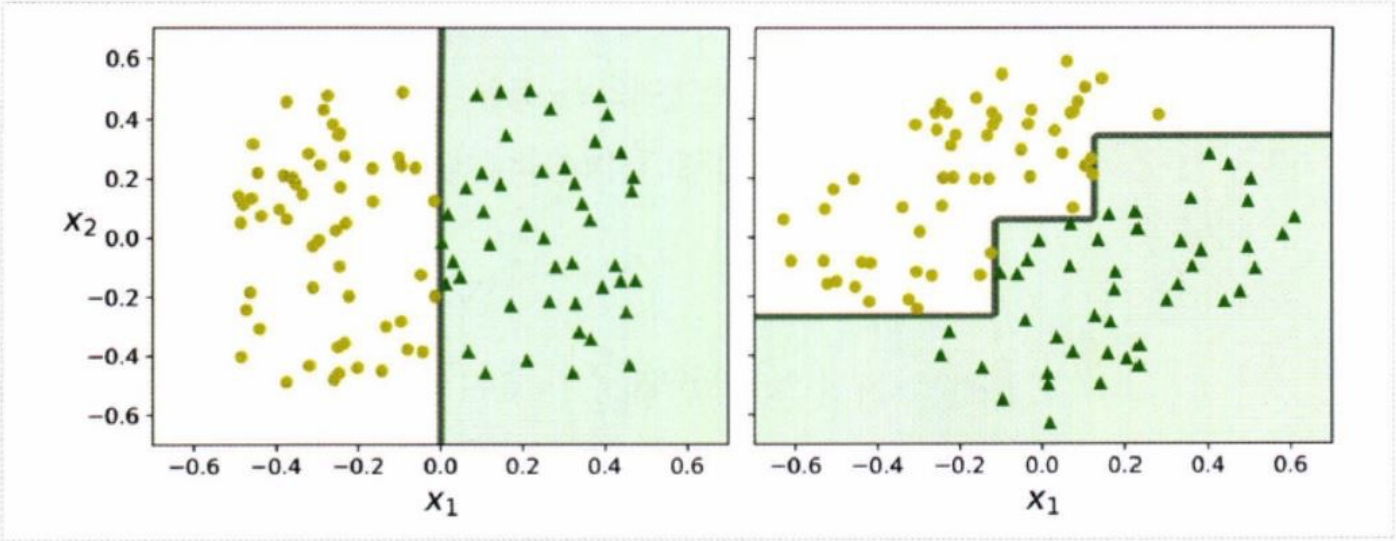
규제가 없을 때 과대적합된 경우(왼쪽)와 규제가 있을 때 과대적합 위험이 감소된 경우(오른쪽)

결정트리 – 불안정성

Decision Tree

불안정성

- 결정 트리는 이해하고 해석하기 쉬우며, 사용하기 편하고, 성능도 뛰어나다.
- 하지만 결정 트리는 계단 모양의 결정 경계를 만들기 때문에, 훈련 세트의 회전에 민감하다.
  - 아래의 그림과 같이 데이터 셋을 45도 회전시키면, 결정 트리 모델이 불필요하게 구불구불해지는 것을 확인할 수 있다.
  - 이러한 경우의 결정 트리 모델은 잘 일반화될 수 없다.

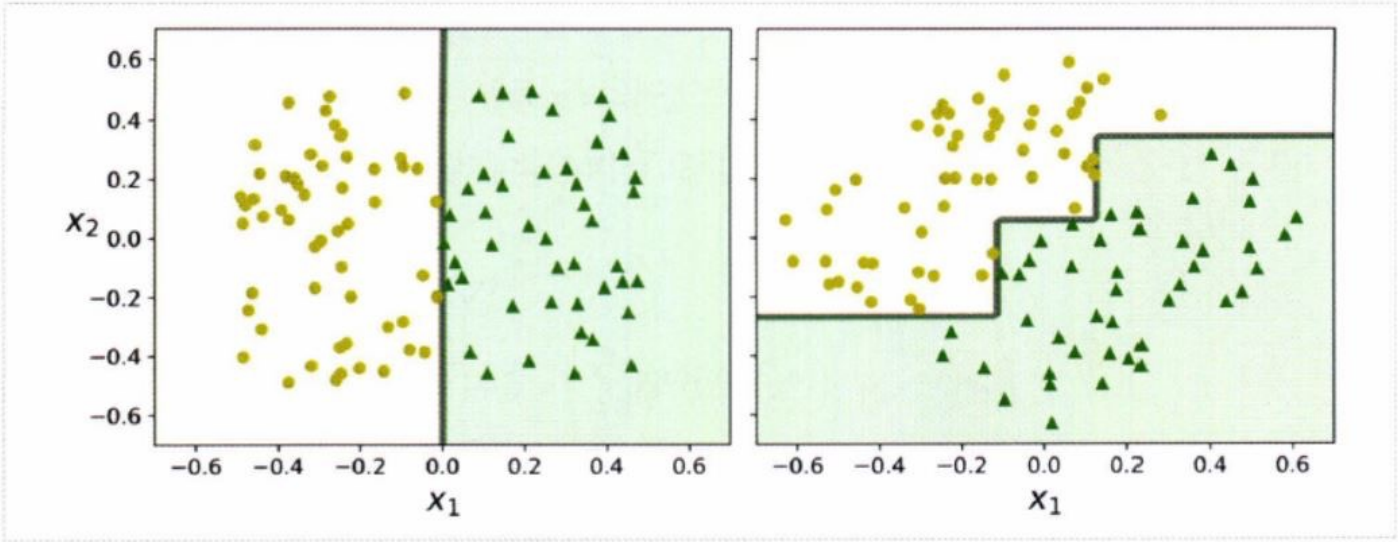


결정트리 – 불안정성

Decision Tree

불안정성 개선 방법

- 이런 문제를 해결하는 한 가지 방법은 훈련 데이터를 더 좋은 방향으로 회전시키는 PCA(주성분 분석) 기법을 사용하는 것이다.
- 또한 다음에 배울 랜덤 포레스트는 많은 트리에서 만든 예측을 평균을 내는 방법으로 이런 불안정성을 극복할 수 있다.



Q&A

질의응답 (5분)

2부: 머신러닝 알고리즘 응용

1) 앙상블

2) 배깅과 랜덤포레스트

3) 부스팅

4) TOP 부스팅 알고리즘

(XGB, LGB, CatGB, NGB)

앙상블 학습

Ensemble Learning

앙상블 학습

- **여러 개의 분류기**를 생성하고, **그 예측을 결합**함으로써 **보다 정확한 최종 예측을 도출**하는 기법이다.
- 각 분류기가 weak learner(약한 학습기)일지라도, **충분하게 많고 다양하다면** 앙상블은 strong learner(강한 학습기)가 될 수 있다.
    - 위처럼 말할 수 있는 근거가 바로 그 유명한 "대수의 법칙"이다.
- 앙상블 방법은 **모든 분류기가 완벽하게 독립적**이고, **오차에 상관관계가 없어야 최고의 성능을 발휘**한다.
    - 따라서, 다양한 분류기를 얻는 한 가지 방법은 **각기 다른 알고리즘으로 학습시키는 것**이다.
- 대표적인 앙상블 모델로는 랜덤 포레스트(결정 트리의 앙상블)가 있다.
- 앙상블 학습의 유형은 전통적으로 **보팅(Voting), 배깅(Bagging), 부스팅(Boosting)**의 세 가지로 나뉜다.

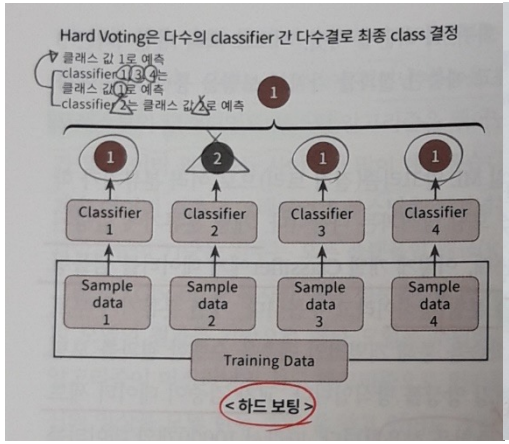
앙상블 학습 - Voting

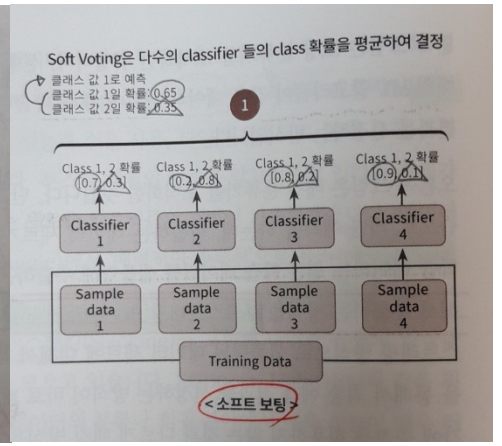
Ensemble Learning

Voting 기반 분류기

- 더 좋은 분류기를 만드는 방법은 각 분류기의 예측을 모아서, 가장 많이 선택된 클래스를 예측하는 것이다.
- Scikit-learn의 VotingClassifier 클래스를 사용하면 된다.

1. Hard Voting 분류기
    - 다수결의 원칙과 유사하다. 즉, 다수결 투표로 정해지는 분류기이다.
2. Soft Voting 분류기
    - 분류기들의 클래스 값 결정 확률을 모두 더하고 이를 평균해서,
      이들 중 확률이 가장 높은 클래스 값을 최종 voting 결과값으로 선정하는 분류기이다.
    - 일반적으로 Hard Voting 방법보다 Soft Voting 방법을 더 많이 사용한다.





Bagging and the Random Forest

In 1906, the statistician Sir Francis Galton was visiting a county fair in England, at which a contest was being held to guess the dressed weight of an ox that was on exhibit. There were 800 guesses, and while the individual guesses varied widely, both the mean and the median came out within 1% of the ox's true weight. James Surowiecki has explored this phenomenon in his book *The Wisdom of Crowds* (Doubleday, 2004). This principle applies to predictive models as well: averaging (or taking majority votes) of multiple models—an *ensemble* of models—turns out to be more accurate than just selecting one model.

Key Terms for Bagging and the Random Forest

Ensemble
Forming a prediction by using a collection of models.

Synonym
Model averaging

Bagging
A general technique to form a collection of models by bootstrapping the data.

Synonym
Bootstrap aggregation

Random forest
A type of bagged estimate based on decision tree models.

Synonym
Bagged decision trees

Variable importance
A measure of the importance of a predictor variable in the performance of the model.

The ensemble approach has been applied to and across many different modeling methods, most publicly in the Netflix Prize, in which Netflix offered a $1 million prize to any contestant who came up with a model that produced a 10% improvement in predicting the rating that a Netflix customer would award a movie. The simple version of ensembles is as follows:

1. Develop a predictive model and record the predictions for a given data set.
2. Repeat for multiple models on the same data.
3. For each record to be predicted, take an average (or a weighted average, or a majority vote) of the predictions.

Ensemble methods have been applied most systematically and effectively to decision trees. Ensemble tree models are so powerful that they provide a way to build good predictive models with relatively little effort.

Going beyond the simple ensemble algorithm, there are two main variants of ensemble models: *bagging* and *boosting*. In the case of ensemble tree models, these are referred to as *random forest* models and *boosted tree* models. This section focuses on bagging; boosting is covered in "Boosting" on page 270.

배깅과 랜덤 포레스트 > Bagging

Bagging and the Random Forest

Bagging

Bagging, which stands for "bootstrap aggregating," was introduced by Leo Breiman in 1994. Suppose we have a response Y and P predictor variables $\mathbf{X} = X_1, X_2, \cdots, X_P$ with N records.

Bagging is like the basic algorithm for ensembles, except that, instead of fitting the various models to the same data, each new model is fitted to a bootstrap resample. Here is the algorithm presented more formally:

1. Initialize M, the number of models to be fit, and n, the number of records to choose ($n < N$). Set the iteration $m = 1$.
2. Take a bootstrap resample (i.e., with replacement) of n records from the training data to form a subsample Y_m and $\mathbf{X}_m$ (the bag).
3. Train a model using Y_m and $\mathbf{X}_m$ to create a set of decision rules $\hat{f}_m(\mathbf{X})$.
4. Increment the model counter $m = m + 1$. If $m <= M$, go to step 2.

In the case where $\hat{f}_m$ predicts the probability $Y = 1$, the bagged estimate is given by:

$$\hat{f} = \frac{1}{M}\left(\hat{f}_1(\mathbf{X}) + \hat{f}_2(\mathbf{X}) + \cdots + \hat{f}_M(\mathbf{X})\right)$$

앙상블 학습 – Bagging & Pasting

Ensemble Learning

Bagging = Bootstrapping 분할

- 같은 알고리즘으로 여러 개의 분류기를 만들어서 Soft Voting으로 최종 결정하는 알고리즘이다.
  - 즉, **훈련 세트의 subset**을 **무작위로 구성하여 분류기를 각기 다르게 학습**시키는 것이다.
- 훈련 세트에서 **중복을 허용하여 샘플링**하는 방식이다.
  (통계학에서는 중복을 허용한 리샘플링(resampling)을 부트스트래핑(Bootstrapping)이라고 한다.)
- 배깅의 대표적인 알고리즘은 **랜덤 포레스트**이다.

Pasting

- 배깅과 유사하나, 훈련 세트에서 중복을 허용하지 않고 샘플링하는 점이 다르다.

앙상블 학습 – Bagging 분류기 평가방법

Ensemble Learning

OOB 평가 (out-of-bag)

- 배깅(Bagging)을 사용하면 어떤 샘플은 여러 번 샘플링되고, 어떤 샘플은 전혀 선택되지 않을 수 있다.
  - 이처럼 **선택되지 않는 훈련 샘플을 oob(out-of-bag) 샘플**이라고 부른다.
- 예측기가 훈련되는 동안에는 oob 샘플을 사용하지 않으므로, 별도의 검증 세트를 사용하지 않고 obb 샘플을 사용해서 모델을 평가할 수 있다.
  - 앙상블의 평가는 각 예측기의 oob 평가를 평균해서 얻는다.
  - BaggingClassifier 클래스를 만들 때, oob_score = True로 지정하면 훈련이 끝난 후 자동으로 oob 평가를 수행한다.
  - oob 샘플에 대한 결정 함수의 값도 oob_decision_function_ 변수에서 확인 가능하다.
    - 이 결정 함수는 각 훈련 샘플의 클래스 확률을 반환한다.

```
>>> bag_clf = BaggingClassifier(
...     DecisionTreeClassifier(), n_estimators=500,
...     bootstrap=True, n_jobs=-1, oob_score=True)
...
>>> bag_clf.fit(X_train, y_train)
>>> bag_clf.oob_score_
0.90133333333333332
```

앙상블 학습 – Bagging 분류기 특성 샘플링

Ensemble Learning

데이터 X 특성 샘플링 (max_features, bootstrap_feature)

- 샘플링은 max_features, bootstrap_features 두 매개변수로 조절된다.
- 특성 샘플링은 더 다양한 예측기를 만들며, 편향을 늘리는 대신 분산을 낮춘다.
- 이 기법은 고차원의 데이터 셋을 다룰 때 유용하다.

배깅과 랜덤 포레스트 > Random Forest

Bagging and the Random Forest

Random Forest

The *random forest* is based on applying bagging to decision trees, with one important extension: in addition to sampling the records, the algorithm also samples the variables.[4] In traditional decision trees, to determine how to create a subpartition of a partition A, the algorithm makes the choice of variable and split point by minimizing a criterion such as Gini impurity (see "Measuring Homogeneity or Impurity" on page 254). With random forests, at each stage of the algorithm, the choice of variable is limited to a *random subset of variables*. Compared to the basic tree algorithm (see "The Recursive Partitioning Algorithm" on page 252), the random forest algorithm adds two more steps: the bagging discussed earlier (see "Bagging and the Random Forest" on page 259), and the bootstrap sampling of variables at each split:

1. Take a bootstrap (with replacement) subsample from the *records*.
2. For the first split, sample $p < P$ *variables* at random without replacement.
3. For each of the sampled variables $X_{j(1)}, X_{j(2)}, \ldots, X_{j(p)}$, apply the splitting algorithm:
   a. For each value $s_{j(k)}$ of $X_{j(k)}$:
      i. Split the records in partition A, with $X_{j(k)} < s_{j(k)}$ as one partition and the remaining records where $X_{j(k)} \geq s_{j(k)}$ as another partition.
      ii. Measure the homogeneity of classes within each subpartition of A.
   b. Select the value of $s_{j(k)}$ that produces maximum within-partition homogeneity of class.
4. Select the variable $X_{j(k)}$ and the split value $s_{j(k)}$ that produces maximum within-partition homogeneity of class.
5. Proceed to the next split and repeat the previous steps, starting with step 2.
6. Continue with additional splits, following the same procedure until the tree is grown.
7. Go back to step 1, take another bootstrap subsample, and start the process over again.

How many variables to sample at each step? A rule of thumb is to choose $\sqrt{P}$ where P is the number of predictor variables. The package `randomForest` implements the random forest in *R*. The following applies this package to the loan data (see "K-Nearest Neighbors" on page 238 for a description of the data):

```
rf <- randomForest(outcome ~ borrower_score + payment_inc_ratio,
                   data=loan3000)
rf

Call:
 randomForest(formula = outcome ~ borrower_score + payment_inc_ratio,
     data = loan3000)
               Type of random forest: classification
                     Number of trees: 500
No. of variables tried at each split: 1

     OOB estimate of error rate: 39.17%
Confusion matrix:
        default  paid off  class.error
default     873       572   0.39584775
paid off    603       952   0.38778135
```

In *Python*, we use the method `sklearn.ensemble.RandomForestClassifier`:

```
predictors = ['borrower_score', 'payment_inc_ratio']
outcome = 'outcome'

X = loan3000[predictors]
y = loan3000[outcome]

rf = RandomForestClassifier(n_estimators=500, random_state=1, oob_score=True)
rf.fit(X, y)
```

Random Forest

랜덤 포레스트

- 배깅 방법(또는 페이스팅 방법)을 적용한 결정 트리의 앙상블이다.
  - 전형적으로 max_samples를 훈련 세트의 크기로 지정한다.
- 랜덤 포레스트 알고리즘은 트리의 노드를 분할할 때,
  전체 특성 중에서 최적의 특성을 찾는 대신, 무작위로 선택한 특성 후보 중에서 최적의 특성을 찾는 방식이다.
  - 즉, **무작위성**을 더 주입한다.
  - 이 과정에서 **트리를 더욱 다양하게** 만들고, **편향을 늘리는 대신 분산을 낮추어** 전체적으로 더 훌륭한 모델을 만들어낸다.

배깅과 랜덤 포레스트 > Random Forest

Bagging and the Random Forest

By default, 500 trees are trained. Since there are only two variables in the predictor set, the algorithm randomly selects the variable on which to split at each stage (i.e., a bootstrap subsample of size 1).

The *out-of-bag* (*OOB*) estimate of error is the error rate for the trained models, applied to the data left out of the training set for that tree. Using the output from the model, the OOB error can be plotted versus the number of trees in the random forest in *R*:

```
error_df = data.frame(error_rate=rf$err.rate[,'OOB'],
                      num_trees=1:rf$ntree)
ggplot(error_df, aes(x=num_trees, y=error_rate)) +
  geom_line()
```

The `RandomForestClassifier` implementation has no easy way to get out-of-bag estimates as a function of number of trees in the random forest. We can train a sequence of classifiers with an increasing number of trees and keep track of the `oob_score_` values. This method is, however, not efficient:

```
n_estimator = list(range(20, 510, 5))
oobScores = []
for n in n_estimator:
    rf = RandomForestClassifier(n_estimators=n, criterion='entropy',
                                max_depth=5, random_state=1, oob_score=True)
    rf.fit(X, y)
    oobScores.append(rf.oob_score_)
df = pd.DataFrame({ 'n': n_estimator, 'oobScore': oobScores })
df.plot(x='n', y='oobScore')
```

The result is shown in Figure 6-6. The error rate rapidly decreases from over 0.44 before stabilizing around 0.385. The predicted values can be obtained from the `predict` function and plotted as follows in *R*:

```
pred <- predict(rf, prob=TRUE)
rf_df <- cbind(loan3000, pred = pred)
ggplot(data=rf_df, aes(x=borrower_score, y=payment_inc_ratio,
                       shape=pred, color=pred, size=pred)) +
    geom_point(alpha=.8) +
    scale_color_manual(values = c('paid off'='#b8e186', 'default'='#d95f02')) +
    scale_shape_manual(values = c('paid off'=0, 'default'=1)) +
    scale_size_manual(values = c('paid off'=0.5, 'default'=2))
```

In *Python*, we can create a similar plot as follows:

```
predictions = X.copy()
predictions['prediction'] = rf.predict(X)
predictions.head()

fig, ax = plt.subplots(figsize=(4, 4))

predictions.loc[predictions.prediction=='paid off'].plot(
    x='borrower_score', y='payment_inc_ratio', style='.',
    markerfacecolor='none', markeredgecolor='C1', ax=ax)
predictions.loc[predictions.prediction=='default'].plot(
    x='borrower_score', y='payment_inc_ratio', style='o',
    markerfacecolor='none', markeredgecolor='C0', ax=ax)
ax.legend(['paid off', 'default']);
ax.set_xlim(0, 1)
ax.set_ylim(0, 25)
ax.set_xlabel('borrower_score')
ax.set_ylabel('payment_inc_ratio')
```

배깅과 랜덤 포레스트 > Random Forest

Bagging and the Random Forest

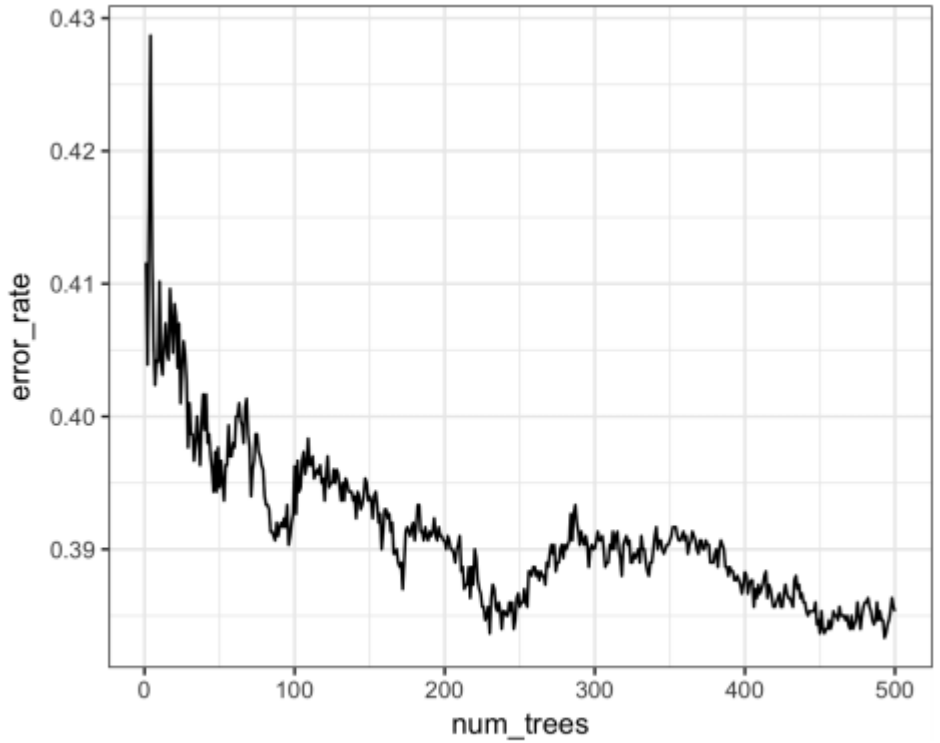


Figure 6-6. An example of the improvement in accuracy of the random forest with the addition of more trees

The plot, shown in Figure 6-7, is quite revealing about the nature of the random forest.

The random forest method is a "black box" method. It produces more accurate predictions than a simple tree, but the simple tree's intuitive decision rules are lost. The random forest predictions are also somewhat noisy: note that some borrowers with a very high score, indicating high creditworthiness, still end up with a prediction of default. This is a result of some unusual records in the data and demonstrates the danger of overfitting by the random forest (see "Bias-Variance Trade-off" on page 247).

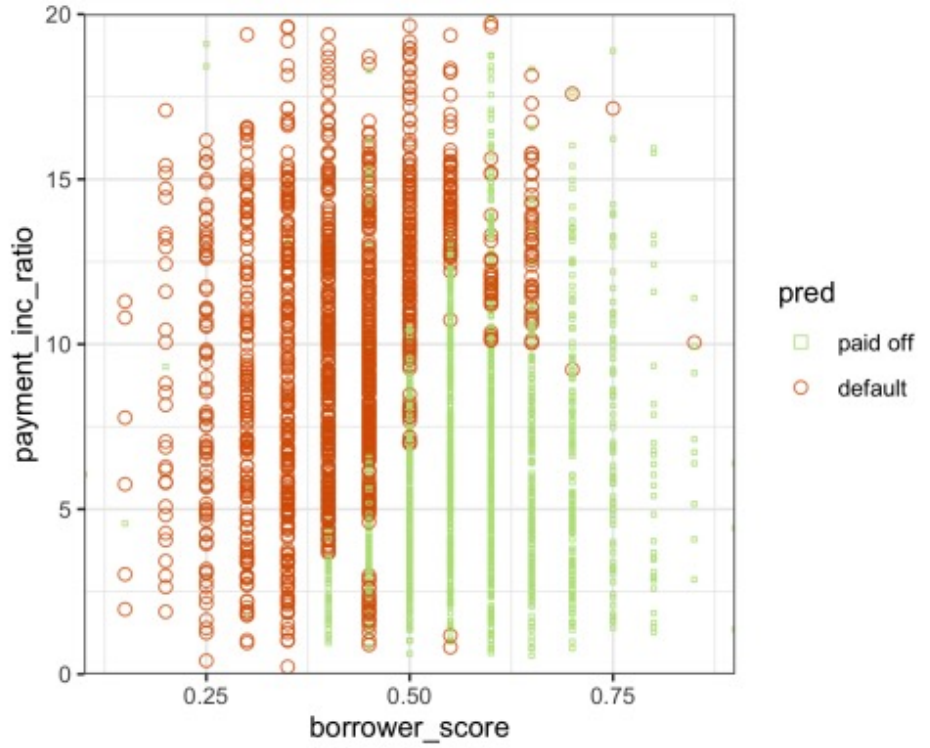


Figure 6-7. The predicted outcomes from the random forest applied to the loan default data

Variable Importance

The power of the random forest algorithm shows itself when you build predictive models for data with many features and records. It has the ability to automatically determine which predictors are important and discover complex relationships between predictors corresponding to interaction terms (see "Interactions and Main Effects" on page 174). For example, fit a model to the loan default data with all columns included. The following shows this in *R*:

```
rf_all <- randomForest(outcome ~ ., data=loan_data, importance=TRUE)
rf_all
Call:
 randomForest(formula = outcome ~ ., data = loan_data, importance = TRUE)
               Type of random forest: classification
                     Number of trees: 500
No. of variables tried at each split: 4

        OOB estimate of  error rate: 33.79%

Confusion matrix:
         paid off default class.error
paid off    14676    7995   0.3526532
default      7325   15346   0.3231000
```

And in *Python*:

```
predictors = ['loan_amnt', 'term', 'annual_inc', 'dti', 'payment_inc_ratio',
              'revol_bal', 'revol_util', 'purpose', 'delinq_2yrs_zero',
              'pub_rec_zero', 'open_acc', 'grade', 'emp_length', 'purpose_',
              'home_', 'emp_len_', 'borrower_score']
outcome = 'outcome'

X = pd.get_dummies(loan_data[predictors], drop_first=True)
y = loan_data[outcome]

rf_all = RandomForestClassifier(n_estimators=500, random_state=1)
rf_all.fit(X, y)
```

The argument `importance=TRUE` requests that the `randomForest` store additional information about the importance of different variables. The function `varImpPlot` will plot the relative performance of the variables (relative to permuting that variable):

```
varImpPlot(rf_all, type=1) ❶
varImpPlot(rf_all, type=2) ❷
```

❶ mean decrease in accuracy

❷ mean decrease in node impurity

In *Python*, the `RandomForestClassifier` collects information about feature importance during training and makes it available with the field `feature_importances_`:

```
importances = rf_all.feature_importances_
```

The "Gini decrease" is available as the `feature_importance_` property of the fitted classifier. Accuracy decrease, however, is not available out of the box for *Python*. We can calculate it (`scores`) using the following code:

```
rf = RandomForestClassifier(n_estimators=500)
scores = defaultdict(list)

# cross-validate the scores on a number of different random splits of the data
for _ in range(3):
    train_X, valid_X, train_y, valid_y = train_test_split(X, y, test_size=0.3)
    rf.fit(train_X, train_y)
    acc = metrics.accuracy_score(valid_y, rf.predict(valid_X))
    for column in X.columns:
        X_t = valid_X.copy()
        X_t[column] = np.random.permutation(X_t[column].values)
        shuff_acc = metrics.accuracy_score(valid_y, rf.predict(X_t))
        scores[column].append((acc-shuff_acc)/acc)
```

배깅과 랜덤 포레스트 > Variable Importance

The result is shown in Figure 6-8. A similar graph can be created with this *Python* code:

```
df = pd.DataFrame({
    'feature': X.columns,
    'Accuracy decrease': [np.mean(scores[column]) for column in X.columns],
    'Gini decrease': rf_all.feature_importances_,
})
df = df.sort_values('Accuracy decrease')

fig, axes = plt.subplots(ncols=2, figsize=(8, 4.5))
ax = df.plot(kind='barh', x='feature', y='Accuracy decrease',
             legend=False, ax=axes[0])
ax.set_ylabel('')

ax = df.plot(kind='barh', x='feature', y='Gini decrease',
             legend=False, ax=axes[1])
ax.set_ylabel('')
ax.get_yaxis().set_visible(False)
```

There are two ways to measure variable importance:

- By the decrease in accuracy of the model if the values of a variable are randomly permuted (`type=1`). Randomly permuting the values has the effect of removing all predictive power for that variable. The accuracy is computed from the out-of-bag data (so this measure is effectively a cross-validated estimate).
- By the mean decrease in the Gini impurity score (see "Measuring Homogeneity or Impurity" on page 254) for all of the nodes that were split on a variable (`type=2`). This measures how much including that variable improves the purity of the nodes. This measure is based on the training set and is therefore less reliable than a measure calculated on out-of-bag data.

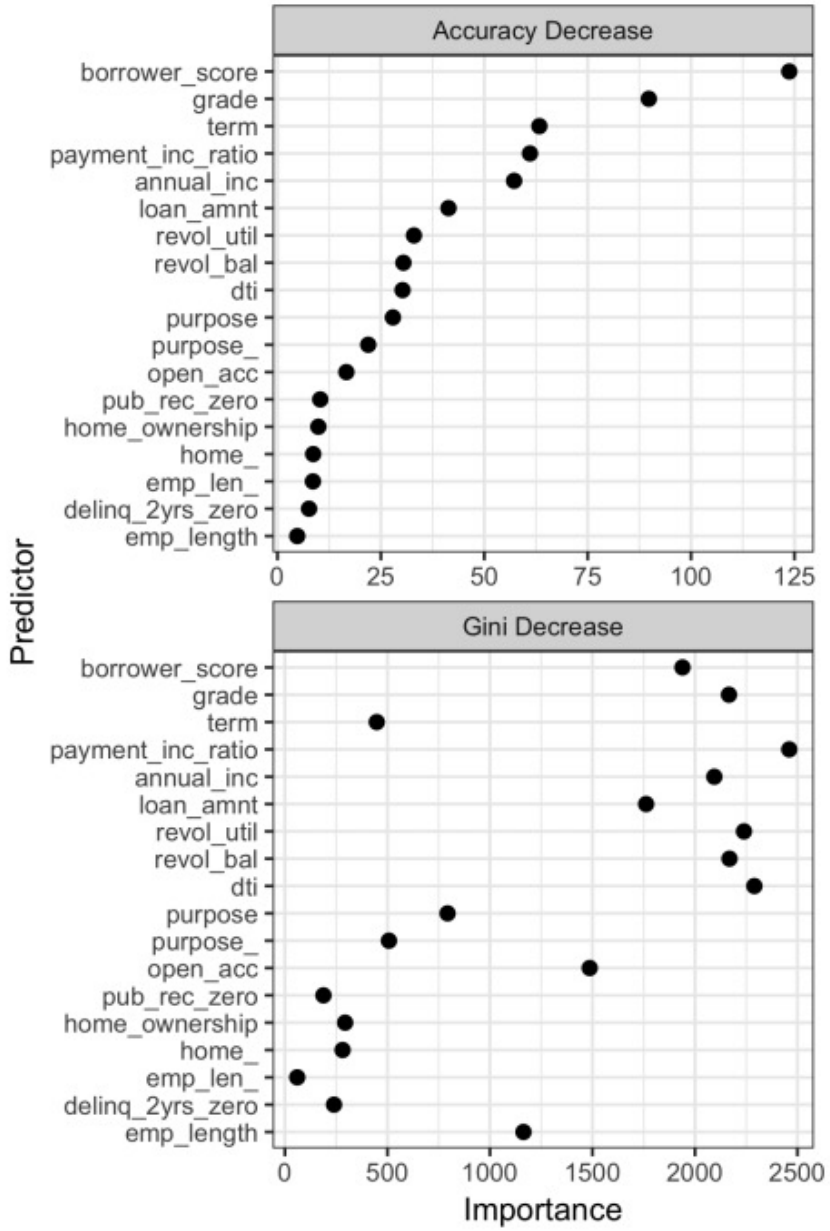


Figure 6-8. The importance of variables for the full model fit to the loan data

배깅과 랜덤 포레스트 > Variable Importance

The top and bottom panels of Figure 6-8 show variable importance according to the decrease in accuracy and in Gini impurity, respectively. The variables in both panels are ranked by the decrease in accuracy. The variable importance scores produced by these two measures are quite different.

Since the accuracy decrease is a more reliable metric, why should we use the Gini impurity decrease measure? By default, `randomForest` computes only this Gini impurity: Gini impurity is a byproduct of the algorithm, whereas model accuracy by variable requires extra computations (randomly permuting the data and predicting this data). In cases where computational complexity is important, such as in a production setting where thousands of models are being fit, it may not be worth the extra computational effort. In addition, the Gini decrease sheds light on which variables the random forest is using to make its splitting rules (recall that this information, readily visible in a simple tree, is effectively lost in a random forest).

특성 중요도

Feature Importance

특성 중요도(Feature Importance)

- 랜덤 포레스트의 장점 중 하나는 특성의 상대적 중요도를 측정하기 쉽다는 것이다.
- Scikit-learn은 어떤 특성을 사용한 노드가 평균적으로 불순도를 얼마나 감소시키는지 확인하여 특성의 중요도를 측정한다.
- 훈련이 끝난 뒤 특성마다 자동으로 이 점수를 계산하고, 특성 중요도의 전체 합이 1이 되도록 결과값을 정규화한다.
    - 해당 결과값은 feature_importances_ 변수에 저장되어 있다.

참고사항

- 결정 트리를 기반으로 하는 모델은 모두 특성 중요도를 제공한다.
    - DecisionTreeClassifier의 특성 중요도는 일부 특성을 완전히 배제시키지만, 무작위성이 주입된 RandomForestClassifier는 거의 모든 특성에 대해 평가할 기회를 가진다.
    - 랜덤 포레스트의 특성 중요도는 각 결정 트리의 특성 중요도를 모두 계산하여 더한 후, 트리 수로 나눈 것이다.

Extra Trees

엑스트라 트리

- 트리를 더욱 무작위하게 만들기 위해,
  최적의 임계값을 찾는 대신 후보 특성을 사용해 무작위로 분할한 다음,
  그 중에서 최상의 분할을 선택하는 방식을 엑스트라 트리 또는 익스트림 랜덤 트리라고 한다.
- 마찬가지로 편향은 늘어나지만 분산은 낮춘다.
- 일반적인 랜덤 포레스트보다 엑스트라 트리가 **훨씬 빠르다.**
  - 모든 노드에서 특성마다 가장 최적의 임계값을 찾을 필요가 없기 때문!

Hyperparameters

The random forest, as with many statistical machine learning algorithms, can be considered a black-box algorithm with knobs to adjust how the box works. These knobs are called *hyperparameters*, which are parameters that you need to set before fitting a model; they are not optimized as part of the training process. While traditional statistical models require choices (e.g., the choice of predictors to use in a regression model), the hyperparameters for random forest are more critical, especially to avoid overfitting. In particular, the two most important hyperparameters for the random forest are:

`nodesize/min_samples_leaf`
: The minimum size for terminal nodes (leaves in the tree). The default is 1 for classification and 5 for regression in *R*. The `scikit-learn` implementation in *Python* uses a default of 1 for both.

`maxnodes/max_leaf_nodes`
: The maximum number of nodes in each decision tree. By default, there is no limit and the largest tree will be fit subject to the constraints of `nodesize`. Note that in *Python*, you specify the maximum number of terminal nodes. The two parameters are related:

$$\text{maxnodes} = 2\text{max\_leaf\_nodes} - 1$$

It may be tempting to ignore these parameters and simply go with the default values. However, using the defaults may lead to overfitting when you apply the random forest to noisy data. When you increase `nodesize/min_samples_leaf` or set `maxnodes/max_leaf_nodes`, the algorithm will fit smaller trees and is less likely to create spurious predictive rules. Cross-validation (see "Cross-Validation" on page 155) can be used to test the effects of setting different values for hyperparameters.

Key Ideas

- Ensemble models improve model accuracy by combining the results from many models.
- Bagging is a particular type of ensemble model based on fitting many models to bootstrapped samples of the data and averaging the models.
- Random forest is a special type of bagging applied to decision trees. In addition to resampling the data, the random forest algorithm samples the predictor variables when splitting the trees.
- A useful output from the random forest is a measure of variable importance that ranks the predictors in terms of their contribution to model accuracy.
- The random forest has a set of hyperparameters that should be tuned using cross-validation to avoid overfitting.

부스팅

Boosting

Boosting

Ensemble models have become a standard tool for predictive modeling. *Boosting* is a general technique to create an ensemble of models. It was developed around the same time as *bagging* (see "Bagging and the Random Forest" on page 259). Like bagging, boosting is most commonly used with decision trees. Despite their similarities, boosting takes a very different approach—one that comes with many more bells and whistles. As a result, while bagging can be done with relatively little tuning, boosting requires much greater care in its application. If these two methods were cars, bagging could be considered a Honda Accord (reliable and steady), whereas boosting could be considered a Porsche (powerful but requires more care).

In linear regression models, the residuals are often examined to see if the fit can be improved (see "Partial Residual Plots and Nonlinearity" on page 185). Boosting takes this concept much further and fits a series of models, in which each successive model seeks to minimize the error of the previous model. Several variants of the algorithm are commonly used: *Adaboost*, *gradient boosting*, and *stochastic gradient boosting*. The latter, stochastic gradient boosting, is the most general and widely used. Indeed, with the right choice of parameters, the algorithm can emulate the random forest.

Key Terms for Boosting

Ensemble
Forming a prediction by using a collection of models.

Synonym
Model averaging

Boosting
A general technique to fit a sequence of models by giving more weight to the records with large residuals for each successive round.

Adaboost
An early version of boosting that reweights the data based on the residuals.

Gradient boosting
A more general form of boosting that is cast in terms of minimizing a cost function.

Stochastic gradient boosting
The most general algorithm for boosting that incorporates resampling of records and columns in each round.

Regularization
A technique to avoid overfitting by adding a penalty term to the cost function on the number of parameters in the model.

Hyperparameters
Parameters that need to be set before fitting the algorithm.

The Boosting Algorithm

There are various boosting algorithms, and the basic idea behind all of them is essentially the same. The easiest to understand is Adaboost, which proceeds as follows:

1. Initialize M, the maximum number of models to be fit, and set the iteration counter $m = 1$. Initialize the observation weights $w_i = 1/N$ for $i = 1, 2, ..., N$. Initialize the ensemble model $\hat{F}_0 = 0$.
2. Using the observation weights $w_1, w_2, ..., w_N$, train a model $\hat{f}_m$ that minimizes the weighted error e_m defined by summing the weights for the misclassified observations.
3. Add the model to the ensemble: $\hat{F}_m = \hat{F}_{m-1} + \alpha_m \hat{f}_m$ where $\alpha_m = \frac{\log 1 - e_m}{e_m}$.
4. Update the weights $w_1, w_2, ..., w_N$ so that the weights are increased for the observations that were misclassified. The size of the increase depends on α_m, with larger values of α_m leading to bigger weights.
5. Increment the model counter $m = m + 1$. If $m \leq M$, go to step 2.

The boosted estimate is given by:

$$\hat{F} = \alpha_1 \hat{f}_1 + \alpha_2 \hat{f}_2 + \cdots + \alpha_M \hat{f}_M$$

By increasing the weights for the observations that were misclassified, the algorithm forces the models to train more heavily on the data for which it performed poorly. The factor α_m ensures that models with lower error have a bigger weight.

Gradient boosting is similar to Adaboost but casts the problem as an optimization of a cost function. Instead of adjusting weights, gradient boosting fits models to a *pseudo-residual*, which has the effect of training more heavily on the larger residuals. In the spirit of the random forest, stochastic gradient boosting adds randomness to the algorithm by sampling observations and predictor variables at each stage.

앙상블 학습 – Boosting

Ensemble Learning

Boosting

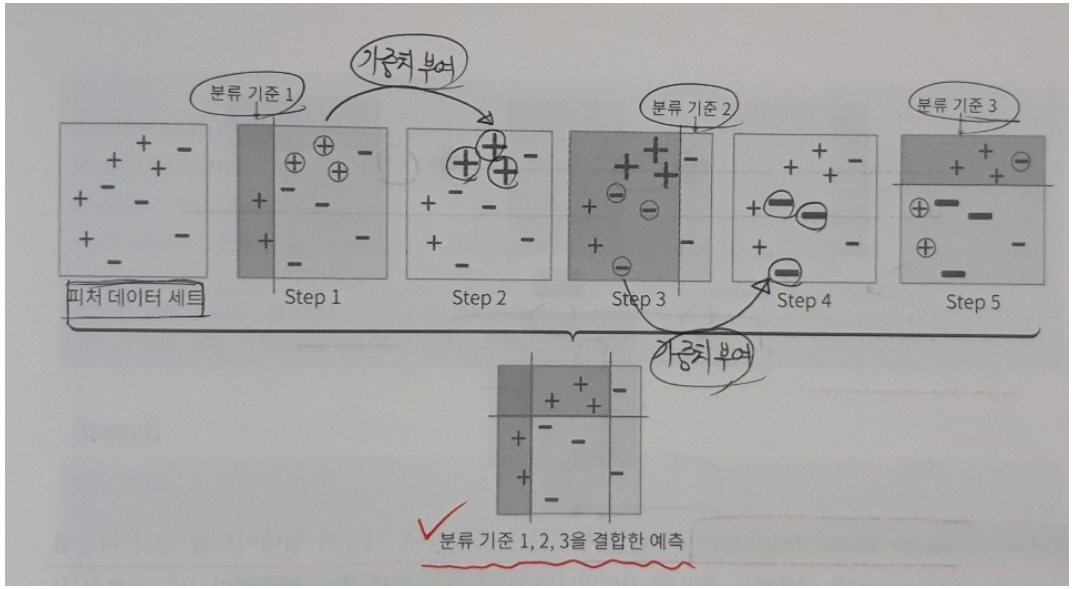
- 여러 개의 약한 학습기(weak learner)를 연결하여 강한 학습기를 만드는 앙상블 방법이다.
  - 즉, **약한 학습기 여러 개를 순차적으로 학습-예측**하면서 **잘못 예측한 데이터에 가중치를 부여**하고, **오류를 개선**해 나가는 학습 방식이다.
- 부스팅의 대표적인 방법으로는 **에이다부스트(AdaBoost)와 그레디언트 부스팅(Gradient Boosting)**이 있다.

앙상블 학습 – AdaBoost

Ensemble Learning

Adaboost

- 이전 모델이 과소적합했던 훈련 샘플의 가중치를 업데이트하면서 순차적으로 학습하는 방식이다.

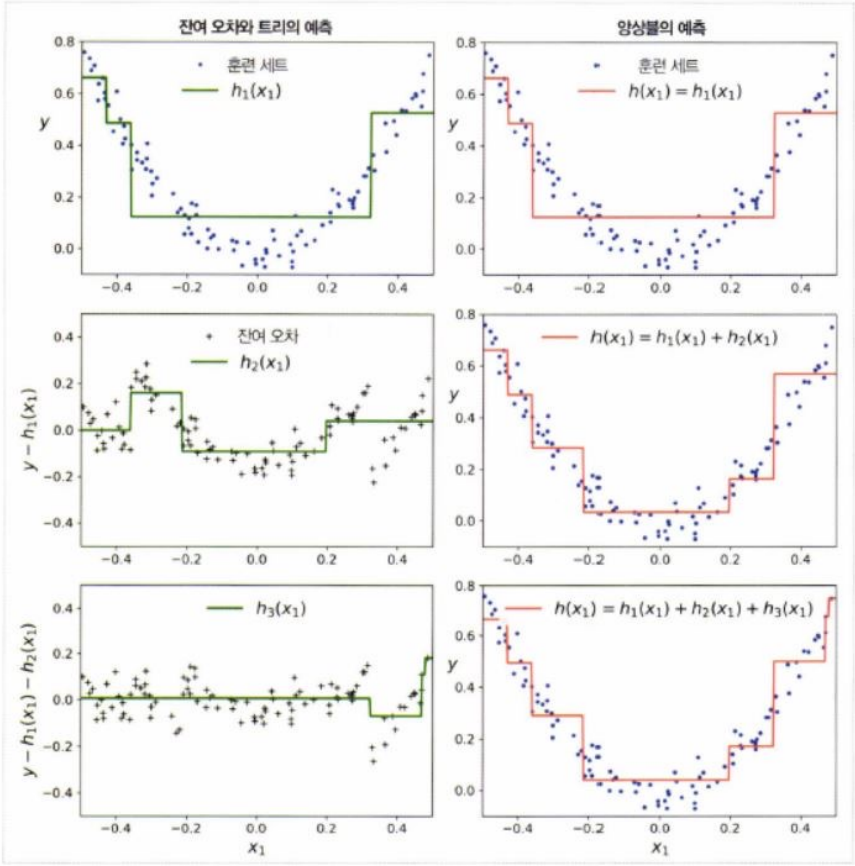


앙상블 학습 – Gradient Boosting

Ensemble Learning

Gradient Boosting

- 마찬가지로 앙상블의 이전까지의 **오차를 보정하도록 예측기를 순차적으로 추가**하는 방식이다.
- 하지만, AdaBoost처럼 **매 반복마다 샘플의 가중치를 수정하지 않고**, 이전 예측기가 만든 **잔여 오차(residual error)에 새로운 예측기를 학습**시킨다.
- 아래의 그림을 보면 알 수 있듯이, 트리가 앙상블에 추가될수록 앙상블의 예측이 점점 좋아지는 것을 확인할 수 있다.



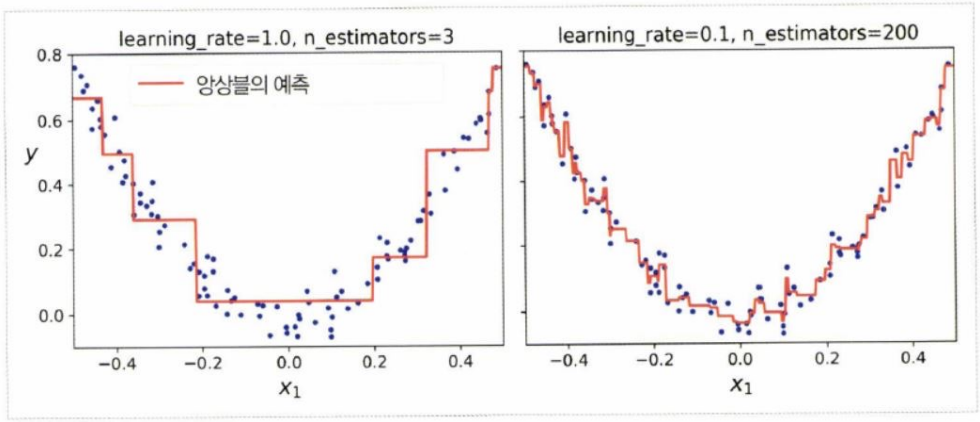
앙상블 학습 – Gradient Boosting

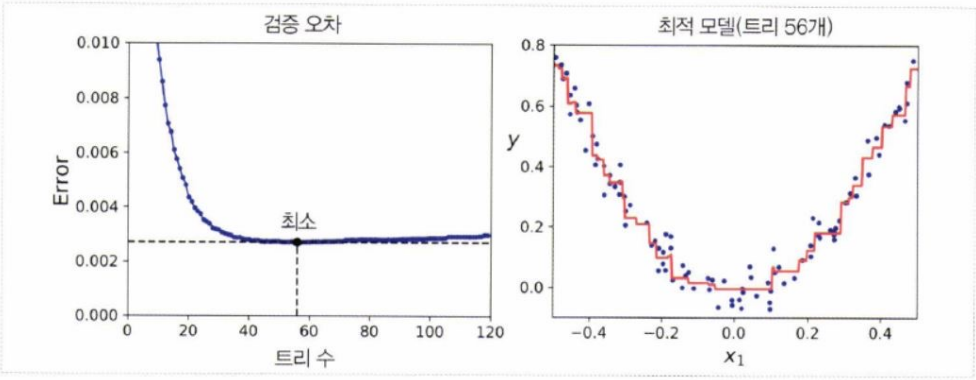
Ensemble Learning

Gradient Boosting 트리 기여도 조절 매개변수

- learning_rate(학습률) 매개변수가 각 트리의 기여 정도를 조절한다.
  - 학습률을 0.1처럼 낮게 설정하면, 앙상블을 훈련 세트에 학습시키기 위해 많은 트리가 필요하지만 예측 성능은 좋아진다.
  - 이는 축소(Shrinkage)라고 부르는 규제 방법이다.

- 최적의 트리 수를 찾기 위해서 조기 종료(Early Stopping) 기법을 사용할 수 있다.
  - staged_predict( ) 매소드를 사용하면 된다.



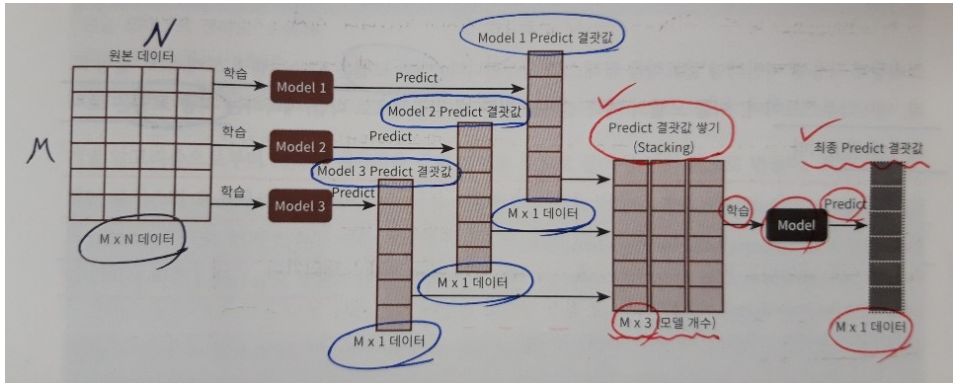


앙상블 학습 – Stacking

Ensemble Learning

Stacking

- 스태킹은 개별적인 **여러 알고리즘을 서로 결합해 예측 결과를 도출**한다는 점에서 배깅(Bagging) 및 부스팅(Boosting)과 유사하다.
- **하지만,** 가장 큰 차이점은 **개별 알고리즘으로 예측한 데이터를 기반으로 다시 예측**을 수행한다는 점이다.
  - 개별 알고리즘의 예측 결과 데이터 세트를 최종적인 메타 데이터 세트로 만든다.
  - 별도의 머신러닝 알고리즘으로 최종 학습을 수행한다.
  - 테스트 데이터를 기반으로 다시 최종 예측(메타 모델을 사용)을 수행한다.
- 정리해보면, 스태킹 모델은 두 종류의 모델이 필요하다.
  - **개별적인 기반 모델** (스태킹을 적용할 때는 많은 개별 모델들이 필요함)
  - 개별 기반 모델의 **예측 데이터를 학습 데이터로 만들어서 학습하는 최종 메타 모델**



- 스태킹 모델은 일반적으로 성능이 비슷한 모델을 결합해서, 좀 더 나은 성능 향상을 도출하기 위해 사용된다.
- Scikit-learn에서는 스태킹을 직접 지원하지 않는다.

XGBoost

The most widely used public domain software for boosting is XGBoost, an implementation of stochastic gradient boosting originally developed by Tianqi Chen and Carlos Guestrin at the University of Washington. A computationally efficient implementation with many options, it is available as a package for most major data science software languages. In *R*, XGBoost is available as the package `xgboost` and with the same name also for *Python*.

The method `xgboost` has many parameters that can, and should, be adjusted (see "Hyperparameters and Cross-Validation" on page 279). Two very important parameters are `subsample`, which controls the fraction of observations that should be sampled at each iteration, and `eta`, a shrinkage factor applied to α_m in the boosting algorithm (see "The Boosting Algorithm" on page 271). Using `subsample` makes boosting act like the random forest except that the sampling is done without replacement. The shrinkage parameter `eta` is helpful to prevent overfitting by reducing the change in the weights (a smaller change in the weights means the algorithm is less likely to overfit to the training set). The following applies `xgboost` in *R* to the loan data with just two predictor variables:

```
predictors <- data.matrix(loan3000[, c('borrower_score', 'payment_inc_ratio')])
label <- as.numeric(loan3000[,'outcome']) - 1
xgb <- xgboost(data=predictors, label=label, objective="binary:logistic",
               params=list(subsample=0.63, eta=0.1), nrounds=100)
[1]     train-error:0.358333
[2]     train-error:0.346333
[3]     train-error:0.347333
...
[99]    train-error:0.239333
[100]   train-error:0.241000
```

Note that `xgboost` does not support the formula syntax, so the predictors need to be converted to a `data.matrix` and the response needs to be converted to 0/1 variables. The `objective` argument tells `xgboost` what type of problem this is; based on this, `xgboost` will choose a metric to optimize.

In *Python*, `xgboost` has two different interfaces: a `scikit-learn` API and a more functional interface like in *R*. To be consistent with other `scikit-learn` methods, some parameters were renamed. For example, `eta` is renamed to `learning_rate`; using `eta` will not fail, but it will not have the desired effect:

```
predictors = ['borrower_score', 'payment_inc_ratio']
outcome = 'outcome'

X = loan3000[predictors]
y = loan3000[outcome]

xgb = XGBClassifier(objective='binary:logistic', subsample=0.63)
xgb.fit(X, y)
--
XGBClassifier(base_score=0.5, booster='gbtree', colsample_bylevel=1,
       colsample_bynode=1, colsample_bytree=1, gamma=0, learning_rate=0.1,
       max_delta_step=0, max_depth=3, min_child_weight=1, missing=None,
       n_estimators=100, n_jobs=1, nthread=None, objective='binary:logistic',
       random_state=0, reg_alpha=0, reg_lambda=1, scale_pos_weight=1, seed=None,
       silent=None, subsample=0.63, verbosity=1)
```

The predicted values can be obtained from the `predict` function in *R* and, since there are only two variables, plotted versus the predictors:

```
pred <- predict(xgb, newdata=predictors)
xgb_df <- cbind(loan3000, pred_default = pred > 0.5, prob_default = pred)
ggplot(data=xgb_df, aes(x=borrower_score, y=payment_inc_ratio,
                        color=pred_default, shape=pred_default, size=pred_default)) +
         geom_point(alpha=.8) +
         scale_color_manual(values = c('FALSE'='#b8e186', 'TRUE'='#d95f02')) +
         scale_shape_manual(values = c('FALSE'=0, 'TRUE'=1)) +
         scale_size_manual(values = c('FALSE'=0.5, 'TRUE'=2))
```

The same figure can be created in *Python* using the following code:

```
fig, ax = plt.subplots(figsize=(6, 4))

xgb_df.loc[xgb_df.prediction=='paid off'].plot(
    x='borrower_score', y='payment_inc_ratio', style='.',
    markerfacecolor='none', markeredgecolor='C1', ax=ax)
xgb_df.loc[xgb_df.prediction=='default'].plot(
    x='borrower_score', y='payment_inc_ratio', style='o',
    markerfacecolor='none', markeredgecolor='C0', ax=ax)
ax.legend(['paid off', 'default']);
ax.set_xlim(0, 1)
ax.set_ylim(0, 25)
ax.set_xlabel('borrower_score')
ax.set_ylabel('payment_inc_ratio')
```

The result is shown in Figure 6-9. Qualitatively, this is similar to the predictions from the random forest; see Figure 6-7. The predictions are somewhat noisy in that some borrowers with a very high borrower score still end up with a prediction of default.

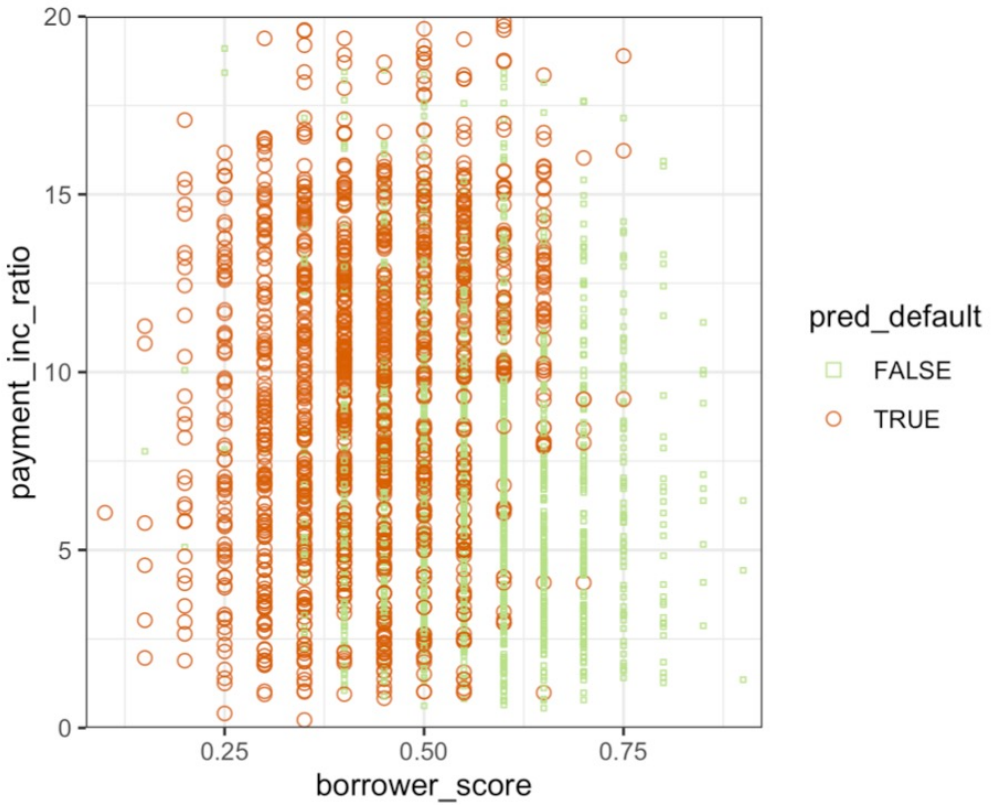


Figure 6-9. The predicted outcomes from XGBoost applied to the loan default data

Regularization: Avoiding Overfitting

Blind application of `xgboost` can lead to unstable models as a result of *overfitting* to the training data. The problem with overfitting is twofold:

- The accuracy of the model on new data not in the training set will be degraded.
- The predictions from the model are highly variable, leading to unstable results.

Any modeling technique is potentially prone to overfitting. For example, if too many variables are included in a regression equation, the model may end up with spurious predictions. However, for most statistical techniques, overfitting can be avoided by a judicious selection of predictor variables. Even the random forest generally produces a reasonable model without tuning the parameters.

This, however, is not the case for `xgboost`. Fit `xgboost` to the loan data for a training set with all of the variables included in the model. In *R*, you can do this as follows:

```
seed <- 400820
predictors <- data.matrix(loan_data[, -which(names(loan_data) %in%
                                       'outcome')])
label <- as.numeric(loan_data$outcome) - 1
test_idx <- sample(nrow(loan_data), 10000)

xgb_default <- xgboost(data=predictors[-test_idx,], label=label[-test_idx],
                       objective='binary:logistic', nrounds=250, verbose=0)
pred_default <- predict(xgb_default, predictors[test_idx,])
error_default <- abs(label[test_idx] - pred_default) > 0.5
xgb_default$evaluation_log[250,]
mean(error_default)
-
iter train_error
1:  250    0.133043

[1] 0.3529
```

We use the function `train_test_split` in *Python* to split the data set into training and test sets:

```
predictors = ['loan_amnt', 'term', 'annual_inc', 'dti', 'payment_inc_ratio',
              'revol_bal', 'revol_util', 'purpose', 'delinq_2yrs_zero',
              'pub_rec_zero', 'open_acc', 'grade', 'emp_length', 'purpose_',
              'home_', 'emp_len_', 'borrower_score']
outcome = 'outcome'

X = pd.get_dummies(loan_data[predictors], drop_first=True)
y = pd.Series([1 if o == 'default' else 0 for o in loan_data[outcome]])

train_X, valid_X, train_y, valid_y = train_test_split(X, y, test_size=10000)

xgb_default = XGBClassifier(objective='binary:logistic', n_estimators=250,
                            max_depth=6, reg_lambda=0, learning_rate=0.3,
                            subsample=1)
xgb_default.fit(train_X, train_y)

pred_default = xgb_default.predict_proba(valid_X)[:, 1]
error_default = abs(valid_y - pred_default) > 0.5
print('default: ', np.mean(error_default))
```

The test set consists of 10,000 randomly sampled records from the full data, and the training set consists of the remaining records. Boosting leads to an error rate of only 13.3% for the training set. The test set, however, has a much higher error rate of 35.3%. This is a result of overfitting: while boosting can explain the variability in the training set very well, the prediction rules do not apply to new data.

부스팅 > Regularization: Avoiding Overfitting

Boosting provides several parameters to avoid overfitting, including the parameters `eta` (or `learning_rate`) and `subsample` (see "XGBoost" on page 272). Another approach is *regularization*, a technique that modifies the cost function in order to *penalize* the complexity of the model. Decision trees are fit by minimizing cost criteria such as Gini's impurity score (see "Measuring Homogeneity or Impurity" on page 254). In `xgboost`, it is possible to modify the cost function by adding a term that measures the complexity of the model.

There are two parameters in `xgboost` to regularize the model: `alpha` and `lambda`, which correspond to Manhattan distance (L1-regularization) and squared Euclidean distance (L2-regularization), respectively (see "Distance Metrics" on page 241). Increasing these parameters will penalize more complex models and reduce the size of the trees that are fit. For example, see what happens if we set `lambda` to 1,000 in *R*:

```
xgb_penalty <- xgboost(data=predictors[-test_idx,], label=label[-test_idx],
                       params=list(eta=.1, subsample=.63, lambda=1000),
                       objective='binary:logistic', nrounds=250, verbose=0)
pred_penalty <- predict(xgb_penalty, predictors[test_idx,])
error_penalty <- abs(label[test_idx] - pred_penalty) > 0.5
xgb_penalty$evaluation_log[250,]
mean(error_penalty)
-
iter train_error
1:  250     0.30966

[1] 0.3286
```

In the `scikit-learn` API, the parameters are called `reg_alpha` and `reg_lambda`:

```
xgb_penalty = XGBClassifier(objective='binary:logistic', n_estimators=250,
                            max_depth=6, reg_lambda=1000, learning_rate=0.1,
                            subsample=0.63)
xgb_penalty.fit(train_X, train_y)
pred_penalty = xgb_penalty.predict_proba(valid_X)[:, 1]
error_penalty = abs(valid_y - pred_penalty) > 0.5
print('penalty: ', np.mean(error_penalty))
```

Now the training error is only slightly lower than the error on the test set.

The `predict` method in *R* offers a convenient argument, `ntreelimit`, that forces only the first *i* trees to be used in the prediction. This lets us directly compare the in-sample versus out-of-sample error rates as more models are included:

```
error_default <- rep(0, 250)
error_penalty <- rep(0, 250)
for(i in 1:250){
  pred_def <- predict(xgb_default, predictors[test_idx,], ntreelimit=i)
  error_default[i] <- mean(abs(label[test_idx] - pred_def) >= 0.5)
  pred_pen <- predict(xgb_penalty, predictors[test_idx,], ntreelimit=i)
  error_penalty[i] <- mean(abs(label[test_idx] - pred_pen) >= 0.5)
}
```

In *Python*, we can call the `predict_proba` method with the `ntree_limit` argument:

```
results = []
for i in range(1, 250):
    train_default = xgb_default.predict_proba(train_X, ntree_limit=i)[:, 1]
    train_penalty = xgb_penalty.predict_proba(train_X, ntree_limit=i)[:, 1]
    pred_default = xgb_default.predict_proba(valid_X, ntree_limit=i)[:, 1]
    pred_penalty = xgb_penalty.predict_proba(valid_X, ntree_limit=i)[:, 1]
    results.append({
        'iterations': i,
        'default train': np.mean(abs(train_y - train_default) > 0.5),
        'penalty train': np.mean(abs(train_y - train_penalty) > 0.5),
        'default test': np.mean(abs(valid_y - pred_default) > 0.5),
        'penalty test': np.mean(abs(valid_y - pred_penalty) > 0.5),
    })

results = pd.DataFrame(results)
results.head()
```

The output from the model returns the error for the training set in the component `xgb_default$evaluation_log`. By combining this with the out-of-sample errors, we can plot the errors versus the number of iterations:

```
errors <- rbind(xgb_default$evaluation_log,
                xgb_penalty$evaluation_log,
                ata.frame(iter=1:250, train_error=error_default),
                data.frame(iter=1:250, train_error=error_penalty))
errors$type <- rep(c('default train', 'penalty train',
                     'default test', 'penalty test'), rep(250, 4))
ggplot(errors, aes(x=iter, y=train_error, group=type)) +
  geom_line(aes(linetype=type, color=type))
```

We can use the `pandas` plot method to create the line graph. The axis returned from the first plot allows us to overlay additional lines onto the same graph. This is a pattern that many of *Python*'s graph packages support:

```
ax = results.plot(x='iterations', y='default test')
results.plot(x='iterations', y='penalty test', ax=ax)
results.plot(x='iterations', y='default train', ax=ax)
results.plot(x='iterations', y='penalty train', ax=ax)
```

The result, displayed in Figure 6-10, shows how the default model steadily improves the accuracy for the training set but actually gets worse for the test set. The penalized model does not exhibit this behavior.

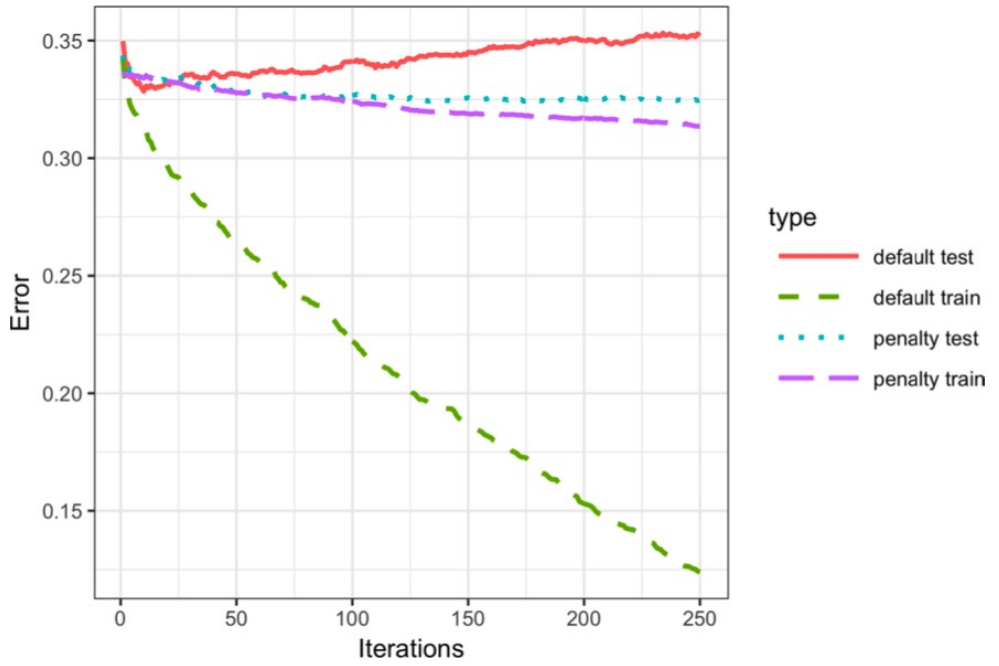


Figure 6-10. The error rate of the default XGBoost versus a penalized version of XGBoost

부스팅 > Regularization: Avoiding Overfitting

Ridge Regression and the Lasso

Adding a penalty on the complexity of a model to help avoid overfitting dates back to the 1970s. Least squares regression minimizes the residual sum of squares (RSS); see "Least Squares" on page 148. *Ridge regression* minimizes the sum of squared residuals plus a penalty term that is a function of the number and size of the coefficients:

$$\sum_{i=1}^{n}\left(Y_i - \hat{b}_0 - \hat{b}_1 X_i - \cdots \hat{b} X_p\right)^2 + \lambda\left(\hat{b}_1^2 + \cdots + \hat{b}_p^2\right)$$

The value of λ determines how much the coefficients are penalized; larger values produce models that are less likely to overfit the data. The *Lasso* is similar, except that it uses Manhattan distance instead of Euclidean distance as a penalty term:

$$\sum_{i=1}^{n}\left(Y_i - \hat{b}_0 - \hat{b}_1 X_i - \cdots \hat{b} X_p\right)^2 + \alpha\left(\left|\hat{b}_1\right| + \cdots + \left|\hat{b}_p\right|\right)$$

The `xgboost` parameters `lambda` (`reg_lambda`) and `alpha` (`reg_alpha`) are acting in a similar manner.

Using Euclidean distance is also known as L2 regularization, and using Manhattan distance as L1 regularization. The `xgboost` parameters `lambda` (`reg_lambda`) and `alpha` (`reg_alpha`) are acting in a similar manner.

Hyperparameters and Cross-Validation

`xgboost` has a daunting array of hyperparameters; see "XGBoost Hyperparameters" on page 281 for a discussion. As seen in "Regularization: Avoiding Overfitting" on page 274, the specific choice can dramatically change the model fit. Given a huge combination of hyperparameters to choose from, how should we be guided in our choice? A standard solution to this problem is to use *cross-validation*; see "Cross-Validation" on page 155. Cross-validation randomly splits up the data into K different groups, also called *folds*. For each fold, a model is trained on the data not in the fold and then evaluated on the data in the fold. This yields a measure of accuracy of the model on out-of-sample data. The best set of hyperparameters is the one given by the model with the lowest overall error as computed by averaging the errors from each of the folds.

To illustrate the technique, we apply it to parameter selection for `xgboost`. In this example, we explore two parameters: the shrinkage parameter `eta` (`learning_rate`—see "XGBoost" on page 272) and the maximum depth of trees `max_depth`. The parameter `max_depth` is the maximum depth of a leaf node to the root of the tree with a default value of six. This gives us another way to control overfitting: deep trees tend to be more complex and may overfit the data. First we set up the folds and parameter list. In *R*, this is done as follows:

```
N <- nrow(loan_data)
fold_number <- sample(1:5, N, replace=TRUE)
params <- data.frame(eta = rep(c(.1, .5, .9), 3),
                     max_depth = rep(c(3, 6, 12), rep(3,3)))
```

Now we apply the preceding algorithm to compute the error for each model and each fold using five folds:

```
error <- matrix(0, nrow=9, ncol=5)
for(i in 1:nrow(params)){
  for(k in 1:5){
    fold_idx <- (1:N)[fold_number == k]
    xgb <- xgboost(data=predictors[-fold_idx,], label=label[-fold_idx],
                   params=list(eta=params[i, 'eta'],
                               max_depth=params[i, 'max_depth']),
                   objective='binary:logistic', nrounds=100, verbose=0)
    pred <- predict(xgb, predictors[fold_idx,])
    error[i, k] <- mean(abs(label[fold_idx] - pred) >= 0.5)
  }
}
```

In the following *Python* code, we create all possible combinations of hyperparameters and fit and evaluate models with each combination:

```
idx = np.random.choice(range(5), size=len(X), replace=True)
error = []
for eta, max_depth in product([0.1, 0.5, 0.9], [3, 6, 9]):  ❶
    xgb = XGBClassifier(objective='binary:logistic', n_estimators=250,
                        max_depth=max_depth, learning_rate=eta)
    cv_error = []
    for k in range(5):
        fold_idx = idx == k
        train_X = X.loc[~fold_idx]; train_y = y[~fold_idx]
        valid_X = X.loc[fold_idx]; valid_y = y[fold_idx]

        xgb.fit(train_X, train_y)
        pred = xgb.predict_proba(valid_X)[:, 1]
        cv_error.append(np.mean(abs(valid_y - pred) > 0.5))
    error.append({
        'eta': eta,
        'max_depth': max_depth,
        'avg_error': np.mean(cv_error)
    })
    print(error[-1])
errors = pd.DataFrame(error)
```

❶ We use the function `itertools.product` from the *Python* standard library to create all possible combinations of the two hyperparameters.

Since we are fitting 45 total models, this can take a while. The errors are stored as a matrix with the models along the rows and folds along the columns. Using the function `rowMeans`, we can compare the error rate for the different parameter sets:

```
avg_error <- 100 * round(rowMeans(error), 4)
cbind(params, avg_error)
  eta max_depth avg_error
1 0.1         3     32.90
2 0.5         3     33.43
3 0.9         3     34.36
4 0.1         6     33.08
5 0.5         6     35.60
6 0.9         6     37.82
7 0.1        12     34.56
8 0.5        12     36.83
9 0.9        12     38.18
```

Cross-validation suggests that using shallower trees with a smaller value of `eta`/`learning_rate` yields more accurate results. Since these models are also more stable, the best parameters to use are `eta=0.1` and `max_depth=3` (or possibly `max_depth=6`).

XGBoost Hyperparameters

The hyperparameters for `xgboost` are primarily used to balance overfitting with the accuracy and computational complexity. For a complete discussion of the parameters, refer to the `xgboost` documentation.

`eta/learning_rate`
: The shrinkage factor between 0 and 1 applied to α in the boosting algorithm. The default is 0.3, but for noisy data, smaller values are recommended (e.g., 0.1). In *Python*, the default value is 0.1.

`nrounds/n_estimators`
: The number of boosting rounds. If `eta` is set to a small value, it is important to increase the number of rounds since the algorithm learns more slowly. As long as some parameters are included to prevent overfitting, having more rounds doesn't hurt.

`max_depth`
: The maximum depth of the tree (the default is 6). In contrast to the random forest, which fits very deep trees, boosting usually fits shallow trees. This has the advantage of avoiding spurious complex interactions in the model that can arise from noisy data. In *Python*, the default is 3.

`subsample` *and* `colsample_bytree`
: Fraction of the records to sample without replacement and the fraction of predictors to sample for use in fitting the trees. These parameters, which are similar to those in random forests, help avoid overfitting. The default is 1.0.

`lambda/reg_lambda` *and* `alpha/reg_alpha`
: The regularization parameters to help control overfitting (see "Regularization: Avoiding Overfitting" on page 274). Default values for *Python* are `reg_lambda=1` and `reg_alpha=0`. In *R*, both values have default of 0.

Key Ideas

- Boosting is a class of ensemble models based on fitting a sequence of models, with more weight given to records with large errors in successive rounds.
- Stochastic gradient boosting is the most general type of boosting and offers the best performance. The most common form of stochastic gradient boosting uses tree models.
- XGBoost is a popular and computationally efficient software package for stochastic gradient boosting; it is available in all common languages used in data science.
- Boosting is prone to overfitting the data, and the hyperparameters need to be tuned to avoid this.
- Regularization is one way to avoid overfitting by including a penalty term on the number of parameters (e.g., tree size) in a model.
- Cross-validation is especially important for boosting due to the large number of hyperparameters that need to be set.

앙상블 학습 – XGB (Extreme Gradient Boosting)

Ensemble Learning

XGB (Extreme Gradient Boosting)

- XGBoost는 매우 빠른 속도와 확장성, 그리고 이식성을 갖추고 있다.
- 아직도 머신러닝 경연 대회에서 많은 우승자들이 사용하는 기법 중 하나이다.
- 특징
  - 병렬처리 가능하다.
  - 가지치기 가능하다.
  - 하드웨어 최적화 가능하다. (Gradient 통계값 캐시 메모리 활용)
  - 규제 (L1/L2, Early Stopping 등)
  - Sparsity 인식 가능하다.
  - Weighted Quantile Sketch 기능으로 최적의 분기점을 찾는데 탁월하다.
  - Cross Validation이 Built-in 되어있다.

앙상블 학습 – LightGBM

Ensemble Learning

LightGBM

- LightGBM의 메인 기술은 **GOSS(Gradient-based One-Side Sampling)**이다.
  - GOSS는 Information gain을 계산할 때 기울기가 작은(가중치가 작은)개체에 승수 상수를 적용하여 데이터를 증폭시킨다.
- 특징
  - 속도 / 성능면에서 XGBoost대비 퍼포먼스가 좋다. (특히 속도)
  - 일반 GBM 모형들은 Level-wise(균형 트리 분할) 방식이지만,
    LightGBM 모형은 Leaf-wise(리프 중심 분할) 방식을 활용하여 깊은 트리를 만들고 소요되는 시간 및 메모리 모두를 절약하였다.
  - 단점
    - 적은 데이터에 대한 Overfitting이 일어나기 쉽다.
      - 여기서 '적다'의 기준은 약 10000건 수준

앙상블 학습 – CatBoost (Categorical Boosting)

Ensemble Learning

CatBoost (Categorical Boosting)

- Catergorical feature를 처리하는데 중점을 둔 알고리즘
- 기존 GBM기반 알고리즘들이 가지고 있는 target leakage 문제와 범주형 변수 처리 문제를,
  ordering principle과 새로운 범주형 변수 처리 방법으로 해결하고자 나왔다.
- 그외 oblivious decision Tree와 feature combination과 같은 방법론도 사용되었다.
  - Olivious descision Tree(망각 결정 트리)는 트리를 분할할 때 동일한 분할 기준이 전체 트리 레벨에서 적용되는 것이다.
    이는 균형적인 트리를 만들 수 있고 overfitting을 막을 수 있다.
  - feature combination은 말그대로 범주형 변수의 조합으로 새로운 변수조합을 만드는 것이다.
- 특징
  - 기존 LightGBM은 매번 boosting round에서 범주형 변수를 gradient statistics를 활용해서 변환하여
    계산시간과 메모리를 많이 잡아 먹었던 것과는 비교된다. (학습은 LightGBM이, 예측 시간은 CatBoost가 가장 빠르다)
  - 단점
    - 데이터 대부분이 수치형일 경우 큰 효과를 내기가 어렵다.

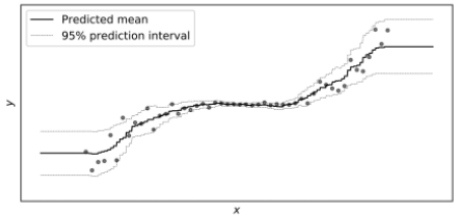
앙상블 학습 – NgBoost (Natural Gradieng Boosting)

Ensemble Learning

NgBoost (Natural Gradient Boosting)

- 학습 이후, 예측 값의 Uncertainty(불확실성)에 대한 통계분석 결과치를 추가로 진행해주는 대표 알고리즘

NGBoost brings predictive uncertainty estimation to Gradient Boosting.



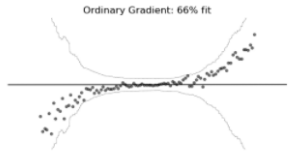
Gradient Boosting methods have generally been among the top performers in predictive accuracy over structured or tabular input data.

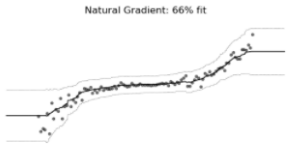
NGBoost enables predictive uncertainty estimation with Gradient Boosting through probabilistic predictions (including real valued outputs). With the use of Natural Gradients, NGBoost overcomes technical challenges that make generic probabilistic prediction hard with gradient boosting.

We release an open source package of our implementation on GitHub.

READ OUR PAPER | GITHUB | DOCUMENTATION

The natural gradient makes learning efficient and effective.





Our key innovation is in employing the natural gradient to perform gradient boosting by casting it as a problem of determining the parameters of a probability distribution.

Ordinary gradients can be highly unsuitable for learning multi-parameter probability distributions (such as the Normal distribution). The training dynamics with the use of natural gradients tends to be much more stable and result in a better fit, as seen in the probabilistic regression example above.

Q&A

질의응답 (5분)

다음 주 배울 내용

Contents for Next Week

비지도 학습

- 주성분분석
- k-평균 클러스터링
- 계층적 클러스터링
- 모델 기반 클러스터링
- 스케일링과 범주형 변수

프로젝트시 체크리스트 (핸즈온 머신러닝 881-886 Page 참고)

Project Checklist

머신러닝 프로젝트 #

끝. 감사합니다.

수업 듣느라 수고하셨습니다.